Fundamentos de Aprendizagem Automática

Class 05 — Support Vector Machines and Neural Networks

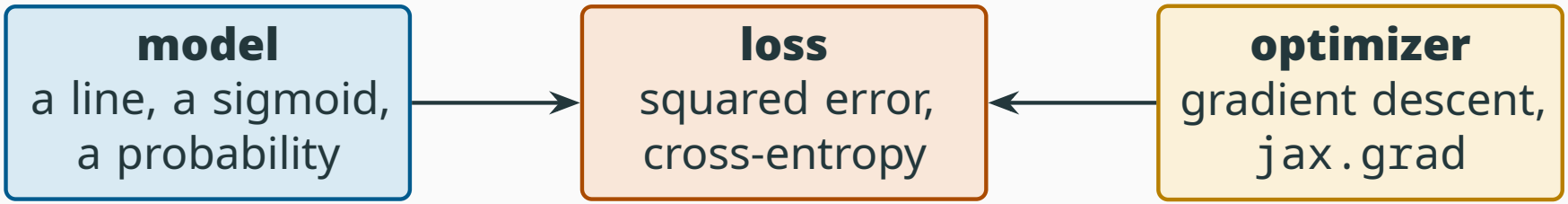
Mário Antunes

Universidade de Aveiro

2026

From Linear Models to Margins

Where We Left Off



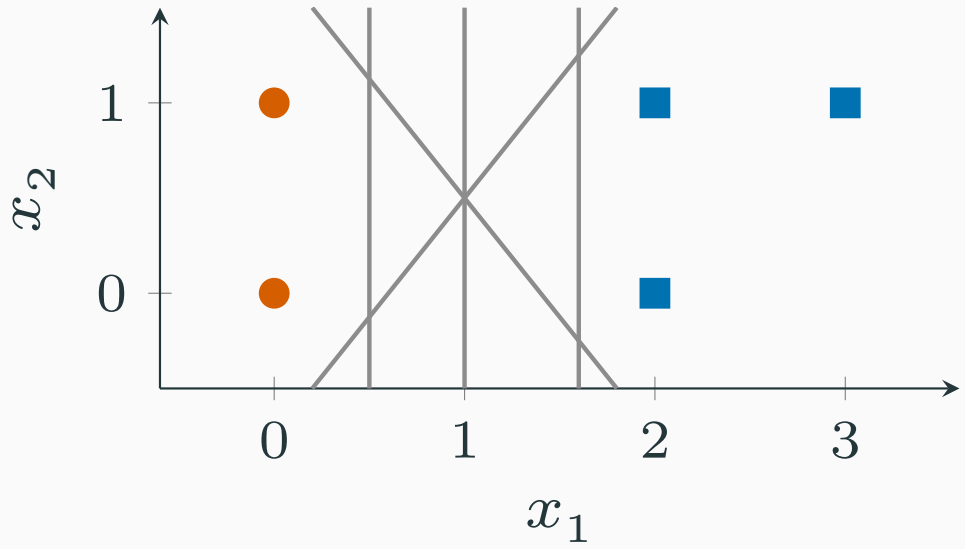
Class 05: two new models, one new loss, one new way to build features

What We Do Today

- **Recap:** every model is a *model* + *a loss* + *an optimizer* (Class 03); every loss is a *negative log-likelihood* (Class 04)
- **So far** the models were *lines*, or lines in a space of hand-made polynomial features
- **Today:**
  - the **best line**: the support vector machine
  - **bending** it without building features by hand: kernels
  - **learning the features** themselves: neural networks
- **Project 1:** the SVM is an *analogizer*, the neural network a *connectionist*

The Problem: Many Lines Fit, Which One?

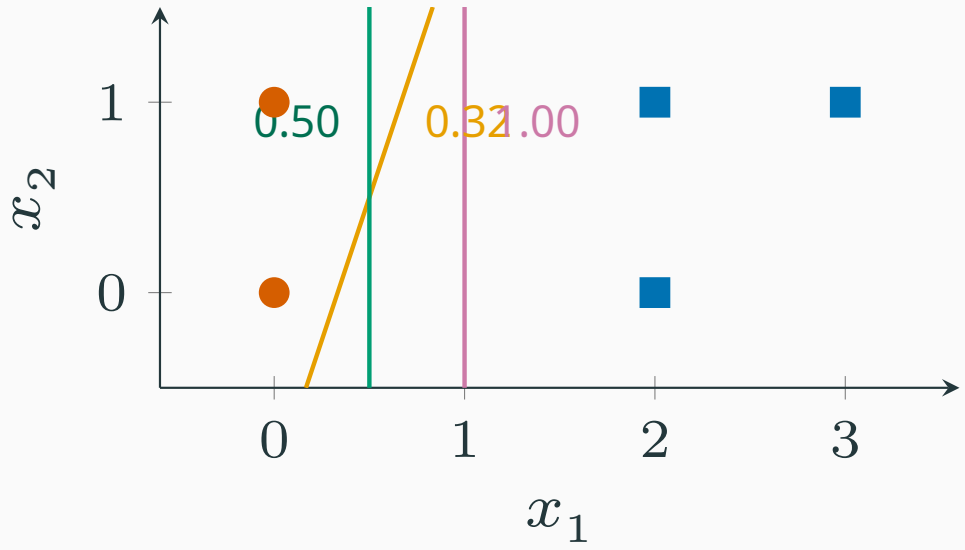
Example 1: five points, two classes (red -1, blue $+1$).



- Many lines separate them: five are drawn
- Each makes **zero** mistakes on these points
- A new point near the boundary gets a different class from each line

The Perceptron: the Line Depends on the Order

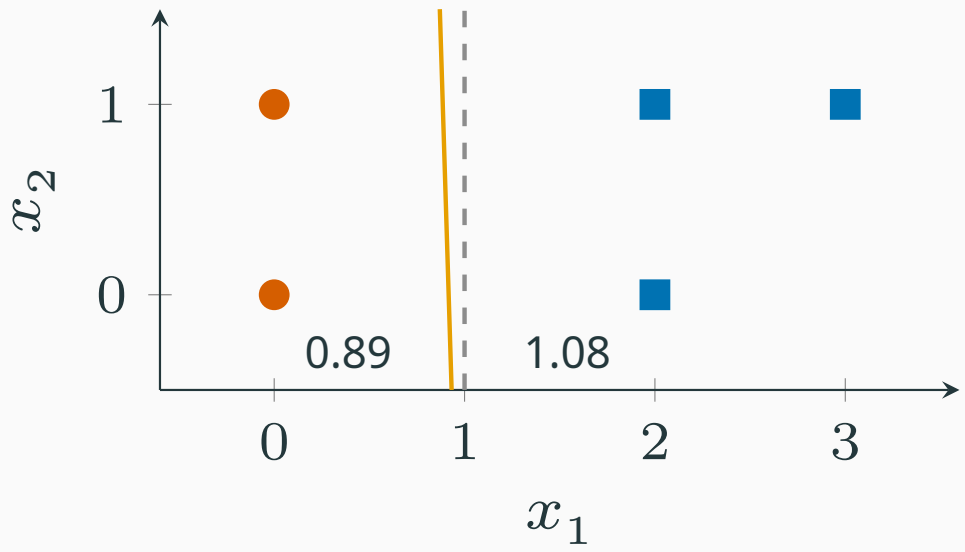
Same data, update $w \leftarrow w + y\,x$, $b \leftarrow b + y$ on every mistake (Class 03):



- The 120 orders of the points give **6 different lines**
- Margins from **0.32** to **1.00** (numbers on the lines)
- 38 orders end at margin ≤ 0.33; only 2 reach 1.00

Logistic Regression: Good, Not the Safest

Logistic regression ($C = 1$): $w = (1.14,\ 0.04)$, $b = -1.04$ (orange).



- **Not centred:** 0.89 from the red points, 1.08 from the blue ones
- Margin **0.89**, the widest street has **1.00** (dashed)
- The loss is never zero: every point, even $(3, 1)$, still pulls the line
- So: is there a **safest** line?

Roadmap of the Class

Part	Question	Answer
Support vector machine	which line, among all that separate?	the one with the **widest margin**
Kernels	what if no line is good enough?	measure *similarity* instead of building features
Neural networks	can the features be learned too?	stack layers of neurons, train by **backpropagation**
Training in practice	how to make it work, and know it worked?	initialization, learning rate, regularization, seeds

- Every part ends with a **live demo** with more experiments than the slides show
- **The practical part** (lab):
  - the **SVM**: build it from its loss with `jax.grad` and compare it with scikit-learn SVC
  - a **small network**: build it in JAX and compare it with scikit-learn `MLPClassifier` and a Keras network (JAX backend)

Notation: Vectors

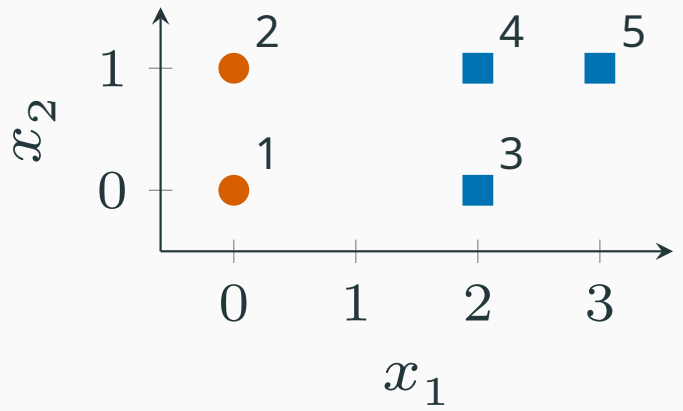
A **feature vector** $x = (x_1, \ldots, x_d)$ lists the d measurements of one example; a **weight vector** $w = (w_1, \ldots, w_d)$ lists how much each one counts.

- **Dot product:** $w^\top x = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$
  - Example: $w = (3, 4)$, $x = (2, 1)$ gives $3 \cdot 2 + 4 \cdot 1 = 10$
- **Length (norm):** $\|w\| = \sqrt{w_1^2 + \cdots + w_d^2}$
  - Example: $\|(3, 4)\| = \sqrt{9 + 16} = 5$
- **Unit vector:** $u = w / \|w\|$ has length 1 and the direction of w

Notation: the Data, the Score and the Prediction

- X: the **data matrix**, $n \times d$, one row x_i per example
- y: the **target vector**, $y_i \in \{-1, +1\}$ (not 0 and 1)
- **Score:** $f(x) = w^\top x + b$; for all examples at once $f = Xw + b$
- **Prediction:** $\hat{y}_i = \text{sign}\, f(x_i)$: $+1$ when $f > 0$, else -1

Example 1 (numbers are the row index):



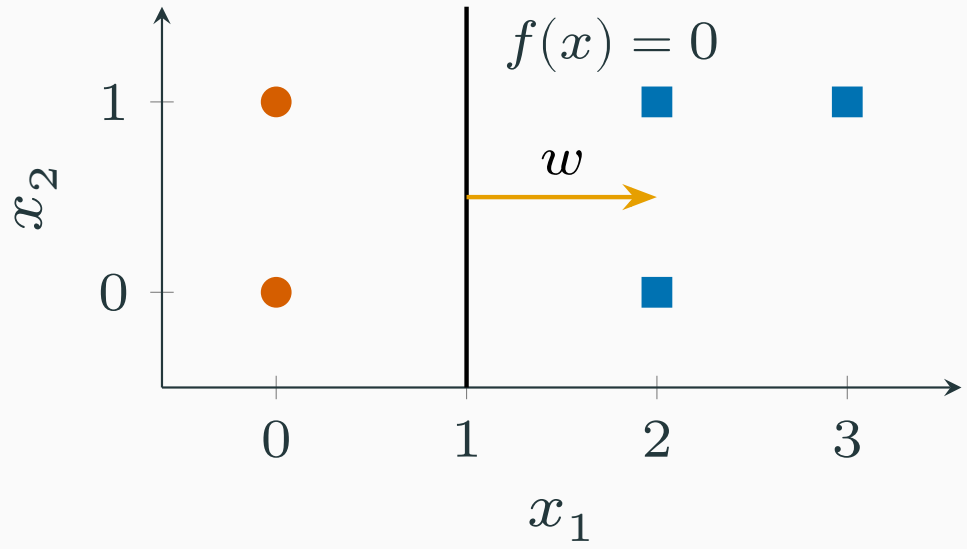
$$X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 2 & 0 \\ 2 & 1 \\ 3 & 1 \end{pmatrix} \qquad y = \begin{pmatrix} -1 \\ -1 \\ +1 \\ +1 \\ +1 \end{pmatrix}$$

$\hat{y} = y$ for $w = (1, 0), b = -1$

Support Vector Machines

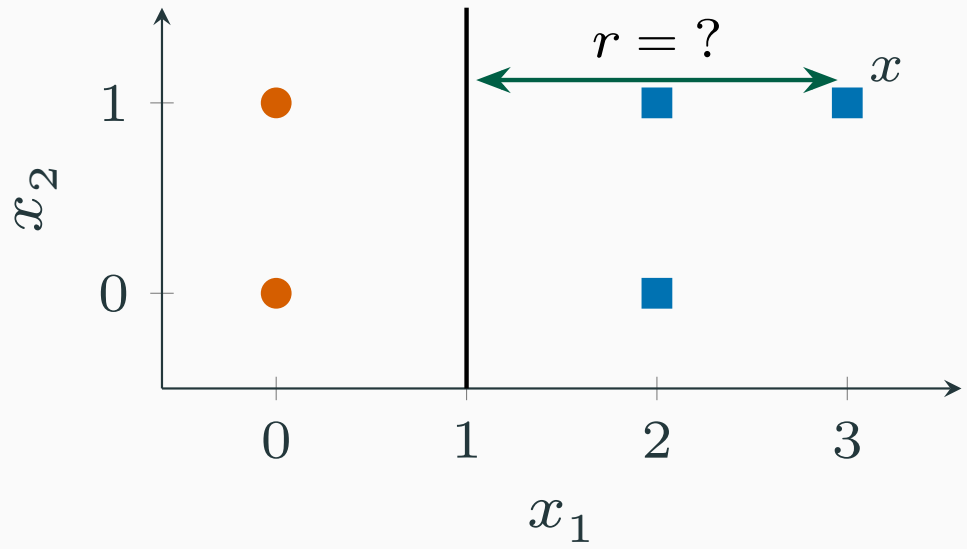
A Line Is an Equation

In two dimensions $f(x) = w_1 x_1 + w_2 x_2 + b$; the **decision boundary** is where $f(x) = 0$: a line (a plane for $d > 2$).



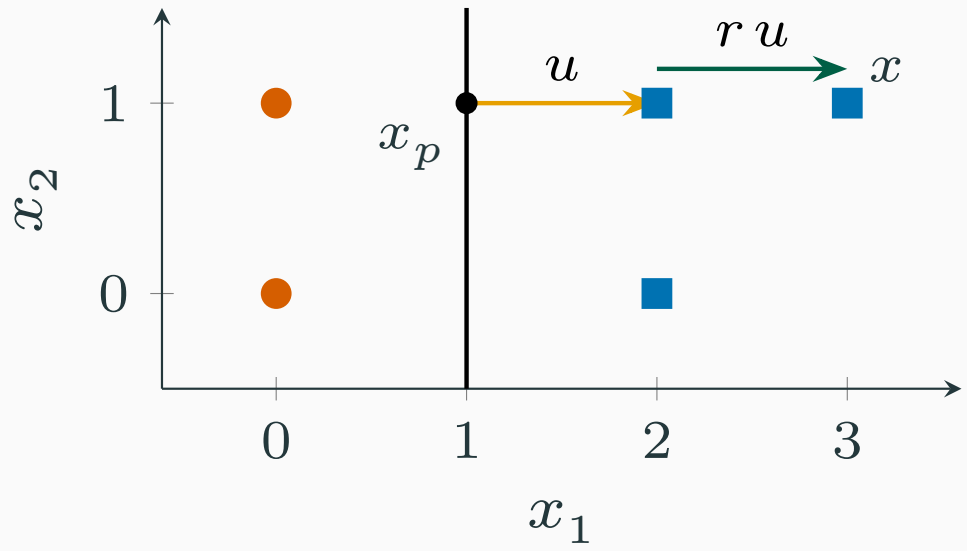
- Take $w = (1, 0), b = -1$: $f(x) = x_1 - 1$, the boundary is $x_1 = 1$
- Scores: $-1, -1$ (red), $+1, +1, +2$ (blue)
- w is **perpendicular** to the line and points to the $+1$ side

How Far Is a Point from the Line?



- $f(x) = w^\top x + b$ is the **linear classifier** of Class 03: the sign of the score is the class
- The score alone is **not a distance**: $(2w, 2b)$ draws the same line and doubles every score
- We want the distance r from x to the line, the same for every scale of (w, b)

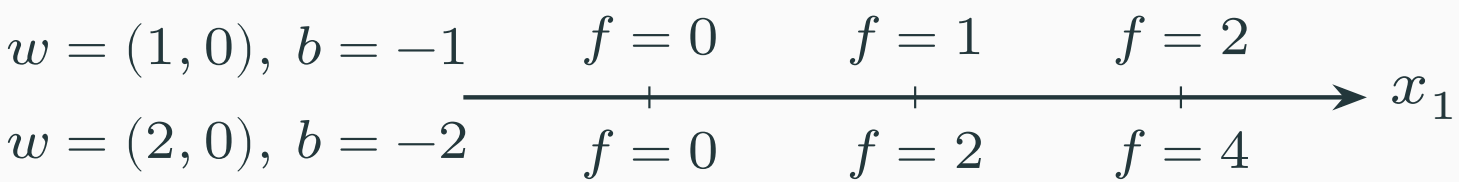
Distance: Build the Point from the Line



1. x_p: the point of the line **closest** to x, so $f(x_p) = 0$
2. The segment $x_p \to x$ is perpendicular to the line, like w: it is **parallel to** w
3. So $x = x_p + r\,u$, with $u = w/\|w\|$ the unit vector and r the signed distance

- Example 1: $x_p = (1, 1)$, $u = (1, 0)$, $r = 2$: $(1, 1) + 2\,(1, 0) = (3, 1)$ ✓

Distance: the Formula



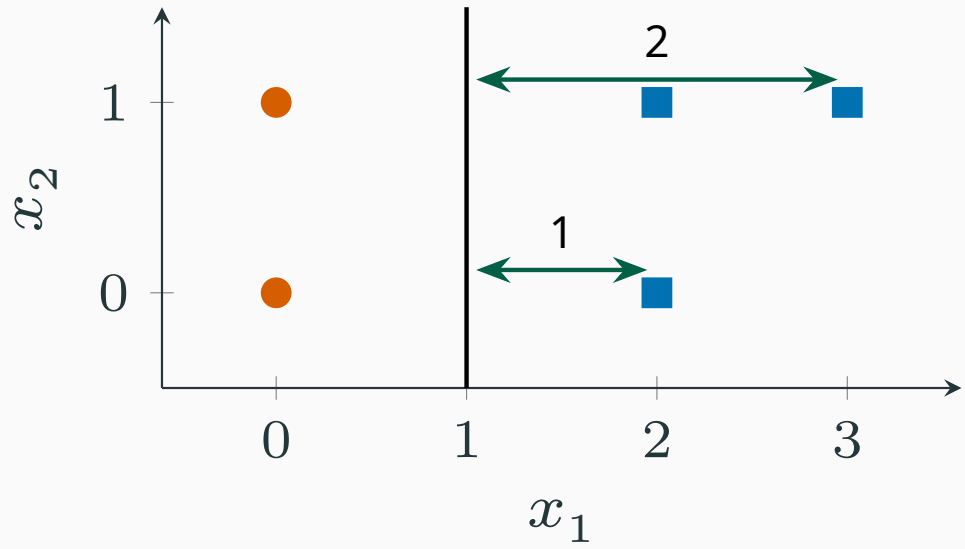
Apply f to $x = x_p + r\,u$; the dot product is linear and $w^\top u = \|w\|$:

$$f(x) = \underbrace{f(x_p)}_{0} + r\,\|w\| \quad \Longrightarrow \quad \boxed{r = \frac{f(x)}{\|w\|}}$$

- Both rulers mark the same line; $f(3, 1)$ is 2 or 4, and $r = 2/1 = 4/2 = 2$ ✓

The Margin of a Point

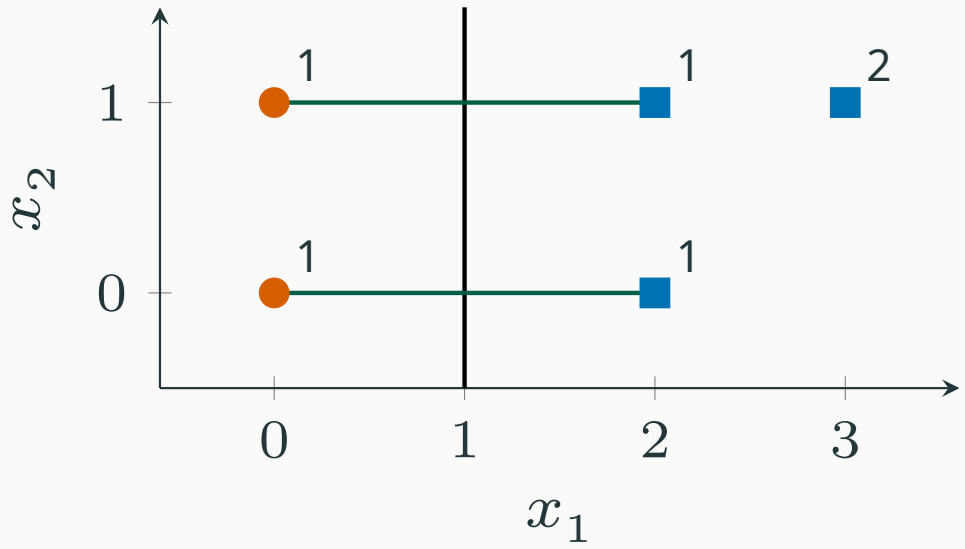
For an example (x_i, y_i):



- **Functional margin** $m_i = y_i \, f(x_i)$: positive when the prediction is **right**, larger when more confident
- **Geometric margin** $m_i / \|w\|$: the true distance, in feature units
- $(3, 1)$: $m = 2$ and distance 2; with $(2w, 2b)$: $m = 4$, distance still 2

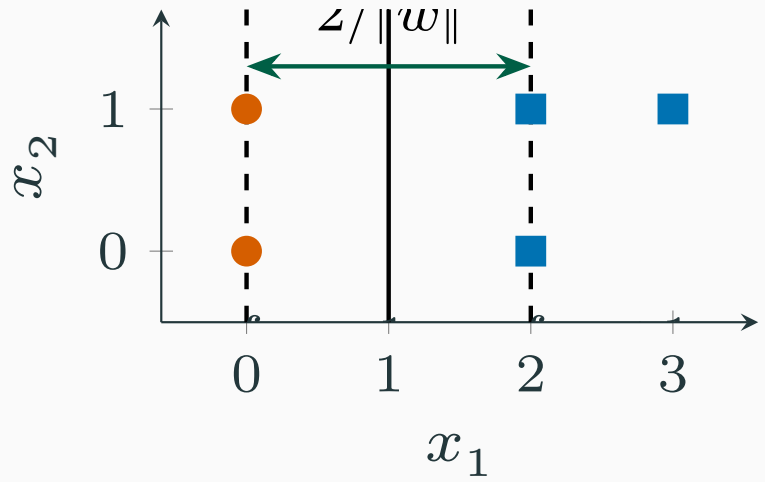
The Margin of the Classifier

Each number is the distance $m_i / \|w\|$ of one point (all $m_i \geq +1$).



- **Margin of the classifier:** the *smallest* geometric margin, $\min_i y_i f(x_i) / \|w\|$: here **1**
- A *negative* margin is a mistake; between 0 and the classifier's margin: "correct but close"
- The SVM looks for the line with the **largest** classifier margin

Maximizing the Margin

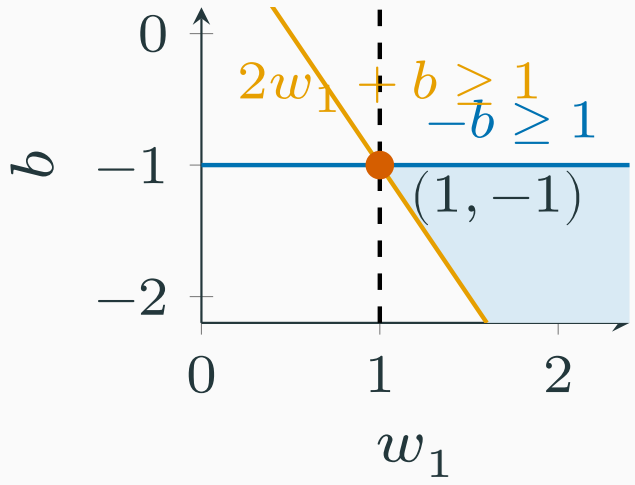


- Rescaling (w, b) does not move the line: **fix the scale** so the closest point has $y_i f(x_i) = 1$
- Then every point has $y_i f(x_i) \geq 1$; the margin is $1/\|w\|$ on each side, the **street** $2/\|w\|$ wide
- A wide street means a **small** $\|w\|$

$$\min_{w,b} \frac{1}{2}\|w\|^2 \qquad \text{subject to} \qquad y_i \left(w^\top x_i + b\right) \geq 1 \quad \text{for every } i$$

Solving Example 1 by Hand

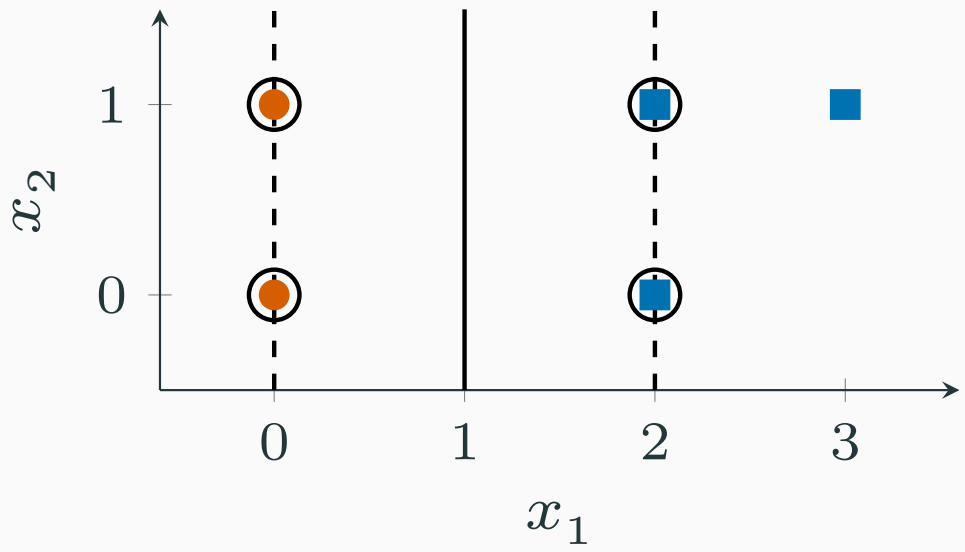
The two groups are symmetric in x_2: the line is vertical, $f(x) = w_1 x_1 + b$.



- Closest points: $-b \geq 1$ (at $x_1 = 0$) and $2w_1 + b \geq 1$ (at $x_1 = 2$)
- Minimize $\frac{1}{2}w_1^2$: the smallest w_1 the constraints allow, both **active**: $b = -1, 2w_1 - 1 = 1, w_1 = 1$

Result: $w = (1, 0), b = -1$, street 2 (SVC agrees).

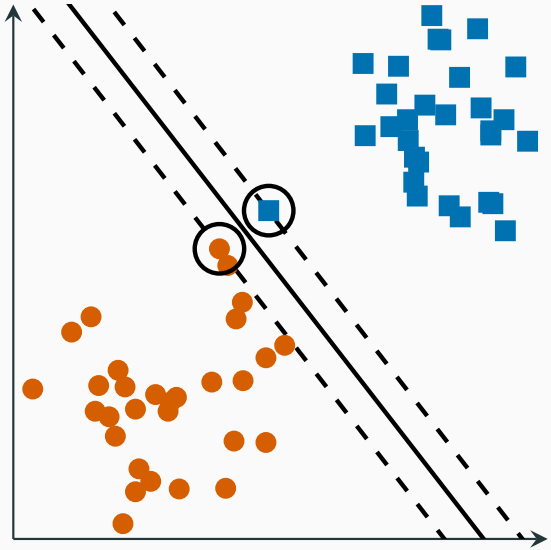
The Maximum-Margin Line



- The line $x_1 = 1$, street 2 wide
- **Support vectors** (circled): the points with $y_i f(x_i) = 1$, here the four at $x_1 = 0$ and $x_1 = 2$
- $(3, 1)$ has margin 2: it could be removed
- A **convex** problem: one global solution, no local minima (Class 02)

Only the Support Vectors Matter (Demo V1)

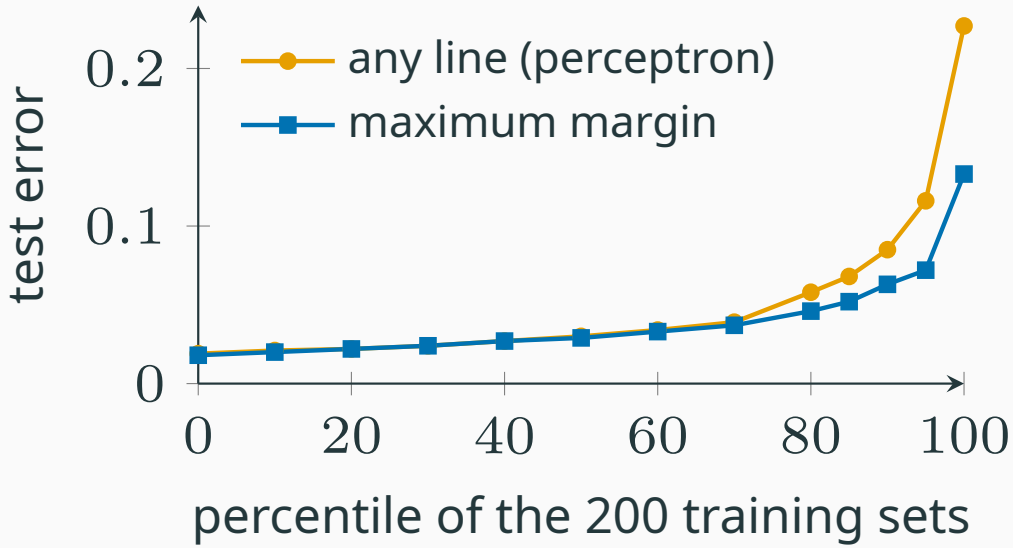
60 points, two blobs; **2** support vectors (circled). Change of w when we:



- retrain on **those two only**: 7×10^{-8} (none)
- push the other 58 points away: 0.0
- move **one support vector** by 0.5: 2.26
- The support vectors **are** the model; the perceptron, instead, moved at every mistake

Why a Large Margin Helps (Demo V1)

A new point is the training source plus noise: a line far from every training point is less likely to be crossed.



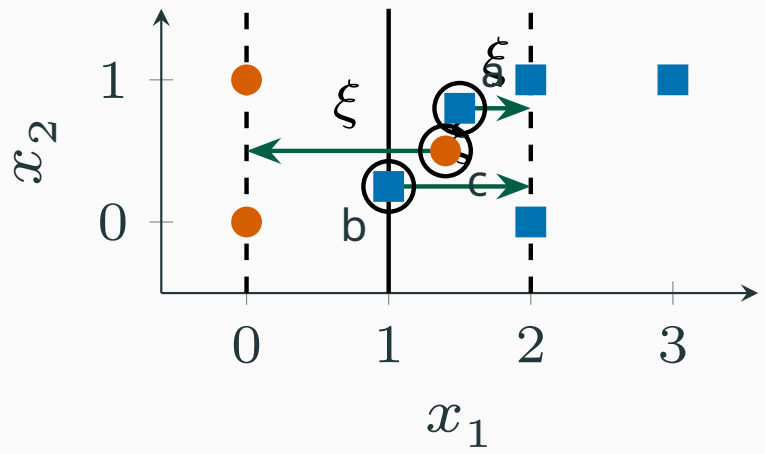
- 200 random training sets of 14 points, 20 000 fresh test points
- Mean error: **0.044** (perceptron), **0.036** (SVM)
- Worst case: **0.227** against **0.133**

Margin and Generalization

- The SVM wins **modestly on average** and clearly in the **bad cases**: the margin protects against an unlucky line
- **Theory** (Vapnik): the error bound depends on R^2/margin^2 (R = radius of the data), **not** on the number of features
  - a wide margin is a form of regularization (Class 03)
- Margins are *relative to the scale of the data*: this is why the features are scaled first

When the Classes Overlap: the Slack

Real classes overlap: no line has all $y_i f(x_i) \geq 1$. We allow a violation and **charge for it**.



- Example 1 plus three new points (rings), same line $x_1 = 1$
- The **slack** ξ_i is how far a point is inside the street or on the wrong side (green arrows):

$$\xi_i = \max\left(0,\ 1 - y_i f(x_i)\right)$$

Slack: Four Situations

situation of the point	point	margin $m = y_i f(x_i)$	slack ξ_i
correct, outside the street	$(3, 1)$	$m = 2 \geq 1$	0
correct, but **inside** the street	a	$0 < m = 0.5 < 1$	$1 - m = 0.5$
on the line	b	$m = 0$	1
wrong side	c	$m = -0.4 < 0$	$1 - m = 1.4$

- The slack is a *cost per point*: it is the **hinge loss** of that point; the total is $0.5 + 1 + 1.4 = 2.9$
- The **support vectors** are the points with $m \leq 1$: slack > 0 or exactly on the street edge

The Soft-Margin Problem

Trade the street width against the total slack with a weight $C > 0$:

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i, \qquad \xi_i = \max(0, 1 - y_i f(x_i))$$

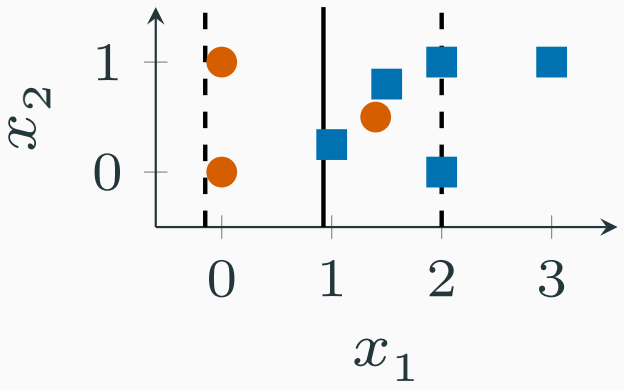
Divide by Cn: the *same* problem is **a loss plus a penalty** (Class 03),

$$\frac{1}{n}\sum_{i=1}^{n}\max\left(0,\ 1 - y_i f(x_i)\right) + \frac{\lambda}{2}\|w\|^2, \qquad \lambda = \frac{1}{Cn}$$

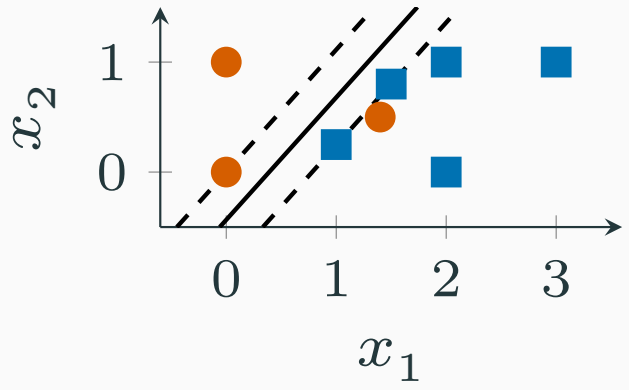
- The mean slack is the **hinge loss**; the penalty is **ridge** with another loss, a Gaussian prior on w (Class 04)

What C Means

$C = 0.3$: street 2.15



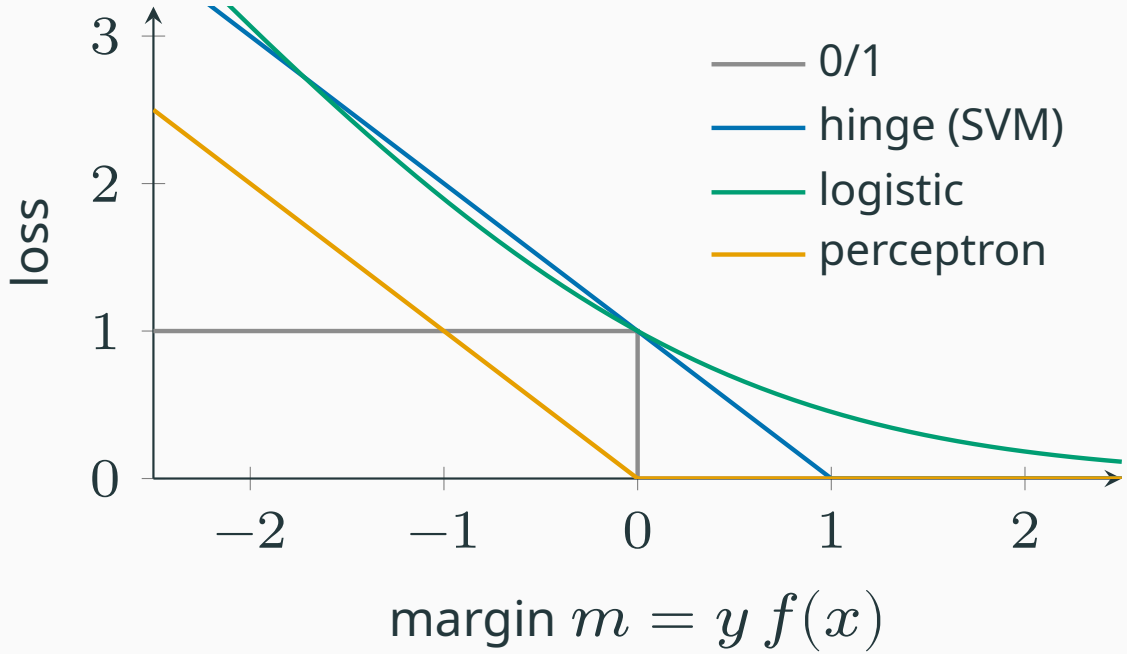
$C = 100$: street 0.58



- C **large:** a violation is expensive, the street is narrow (4 support vectors), the line follows the data: low bias, high variance
- C **small:** violations are cheap, the street is wide (7 of 8 points are support vectors): simple, more bias
- C is a **hyperparameter**: chosen on validation data, never by the training error

The Hinge Loss and Its Relatives

All the losses so far are functions of the margin $m = y\, f(x)$, a different price for each value of m:



Reading the Loss Curves

- **0/1:** counts mistakes; flat everywhere, so no gradient (Class 02)
- **Perceptron:** zero as soon as the sign is right
- **Logistic:** never exactly zero: it always asks for a bit more margin
- **Hinge:** zero for $m \geq 1$: well-classified points are **ignored**, which is why only support vectors matter
- The others are *convex upper bounds* of the 0/1 loss (the logistic in base 2): minimizing them also reduces mistakes

Training 1: Subgradient Descent with `jax.grad`

The hinge has a corner at $m = 1$: there any slope between its two sides will do (a **subgradient**).

For one example, with $m = y\,(w^\top x + b)$:

$$\frac{\partial\,\ell}{\partial w} = \begin{cases} -\,y\,x & m < 1 \quad \text{(inside the street or wrong)} \\ 0 & m > 1 \quad \text{(ignored)} \end{cases}$$

- The update $w \leftarrow w - \eta\big(\lambda w - \frac{1}{n}\sum_{m_i<1} y_i x_i\big)$: **push the line away from the violating points**, shrink w
- Like the perceptron rule of Class 03, but the update also fires for points *inside* the street and always shrinks w
- In JAX it is two lines: the loss with `jnp.maximum(0, 1 - y * f)` and `jax.grad`; Adam does the rest

Subgradient Descent Reaches the QP Solution

C	$\max\lvert w_{\text{jax}} - w_{\text{SVC}}\rvert$	same predictions	objective (ours)	objective (SVC)
0.1	0.0000	100%	0.40918	0.40918
1	0.0000	100%	0.38483	0.38483
10	0.0001	100%	0.38190	0.38190

- SVC uses a quadratic-programming solver; gradient descent reaches the **same** solution

Training 2: the Dual View (Why Kernels Will Work)

The classic solver writes w as a **combination of the training points**. One multiplier $\alpha_i \geq 0$ per constraint (hard margin); zero derivatives of the Lagrangian give

$$w = \sum_i \alpha_i \, y_i \, x_i, \qquad \sum_i \alpha_i y_i = 0$$

- Only points with $\alpha_i > 0$ contribute: the **support vectors** (the constraint is active)
- **Example:** points 1 and 3 of Example 1, $x_1 = (0, 0)$, $y_1 = -1$ and $x_3 = (2, 0)$, $y_3 = +1$
  - $\alpha_1 = \alpha_3 = \alpha$, so $w = \alpha(2, 0)$
  - $f(x_3) = 2\alpha - 1 = 1$ gives $\alpha = \frac{1}{2}$: $w = (1, 0)$, $b = -1$ (the SVC dual coefficients are ∓ 0.5)

The Effect of C (Demo V2)

Two overlapping blobs (200 points), linear SVM, test on 4000 new points:

C	street width	support vectors	train accuracy	test accuracy
0.001	8.60	200	0.840	0.814
0.01	3.20	129	0.850	0.838
0.1	2.05	88	0.850	0.840
1	1.75	79	0.850	0.839
10	1.75	79	0.850	0.839
1000	1.73	78	0.850	0.839

- Small C: a very wide street, *every* point is a support vector, the model underfits (0.814)
- Beyond $C \approx 0.1$ nothing changes: the data are not separable, the solution is stable
- The training accuracy barely moves: C **must be chosen on validation data**

Choosing C and the Role of Scaling

Breast-cancer data (569 tumours, 30 features), 10-fold cross-validation, standardized features:

C	10^{-5}	10^{-2} (best)	10^{3}
validation accuracy	0.933	**0.979**	0.951

- A **log axis**: C acts multiplicatively; try $10^{-5}, 10^{-4}, \dots, 10^{3}$; the plateau is wide
- The penalty $\|w\|^2$ treats all features alike: a feature in $[0, 2500]$ next to one in $[0.05, 0.16]$ must be **standardized** (training statistics only, Class 03)
- Linear SVM vs logistic regression (median of 5 splits): breast cancer 0.972 / 0.972; spam 0.944 / 0.942; digits 0.956 / 0.969: **close**

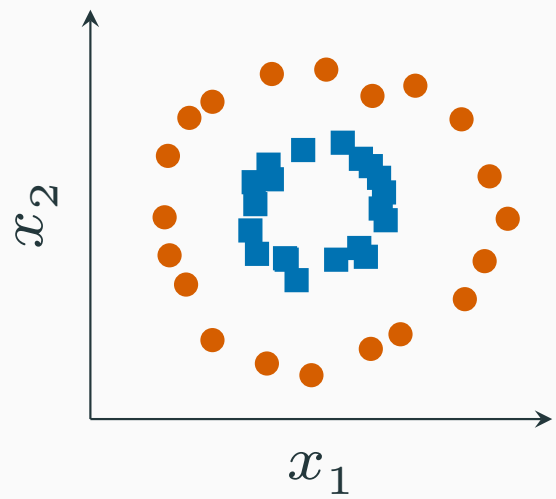
DEMO V1–V2:

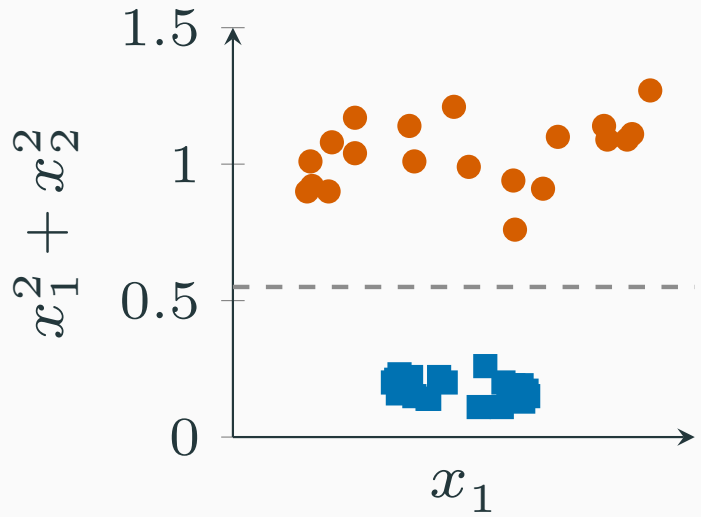
`01_svm.ipynb`

Kernels

When a Line Is Not Enough

Example 2: a ring of one class around a centre of the other. **No** line separates them: a line cuts the plane in two half-planes.





Reading the Lift

- Left: a linear SVM reaches only **0.578** test accuracy on this ring (demo V3)
- Right: add **one feature**, the squared distance to the centre $r^2 = x_1^2 + x_2^2$: the classes are now separated by a *horizontal line*, accuracy **1.000**
- The same linear machine, in a space with a new coordinate: a straight line there is a **circle** in the original plane

Idea: Make New Features, Stay Linear

Class 03 already did it: polynomial features turn a linear model into a curved one (the model is linear in the *parameters*).

- A map φ sends x to a richer vector, for example $\varphi(x_1, x_2) = (x_1,\ x_2,\ x_1^2,\ x_1 x_2,\ x_2^2)$
- Fit a *linear* SVM on $\varphi(x)$: $f(x) = w^\top \varphi(x) + b$

The Price of Building Features

A polynomial map of degree p on d inputs has $\binom{d+p}{p}$ features:

inputs d	degree 2	degree 3
2	6	10
10	66	286
64 (the digits)	2 145	47 905

- The cost explodes; for the *Gaussian* kernel the space is **infinite**: we cannot build $\varphi(x)$
- The dual view saves us: the SVM never needs $\varphi(x)$ itself, only **dot products** of two of them

The Kernel Trick

In the dual the decision function and the training problem use the data only through $x_i^\top x_j$. Replace each dot product by $\varphi(x_i)^\top \varphi(x_j)$ and give it a name:

$$k(a, b) = \varphi(a)^\top \varphi(b)$$

$$f(x) = \sum_{i \in \mathsf{SV}} \alpha_i \, y_i \, k(x_i, x) + b$$

A kernel returns the dot product of two points in a feature space, without building the space.

The Kernel Trick: a Numeric Check

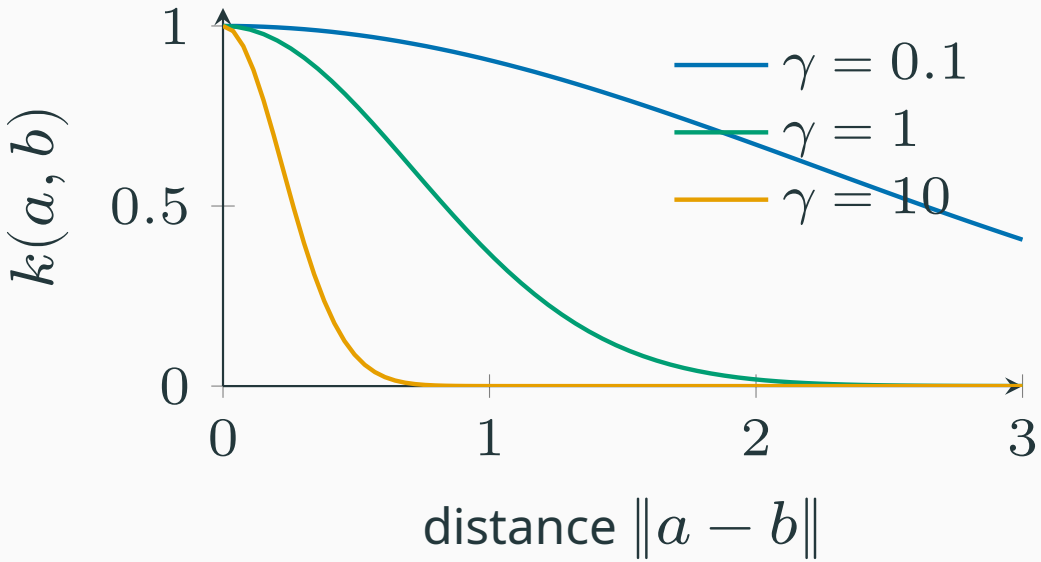
Take $d = 2$, $\varphi(x) = (x_1^2,\ \sqrt{2}\, x_1 x_2,\ x_2^2)$, $k(a, b) = (a^\top b)^2$ and the points $a = (1, 2)$, $b = (3, -1)$:

- Through φ: $\varphi(a) = (1,\ 2\sqrt{2},\ 4)$, $\varphi(b) = (9,\ -3\sqrt{2},\ 1)$, so $\varphi(a)^\top \varphi(b) = 9 - 12 + 4 = 1$
- Directly: $a^\top b = 3 - 2 = 1$, so $(a^\top b)^2 = \mathbf{1}$ ✓: the same number, with one multiplication instead of a map
- Common kernels: **linear** $a^\top b$; **polynomial** $(a^\top b + c_0)^p$; **RBF** $e^{-\gamma \|a-b\|^2}$

The RBF Kernel: Similarity

The most used kernel measures **how close** two points are, on a scale of 0 (far) to 1 (identical):

$$k(a, b) = \exp\left(-\gamma \|a - b\|^2\right)$$



Reading the RBF Kernel

- A prediction is a **weighted vote of the support vectors**, each weighted by its similarity to the new point: a *local* method (an analogizer: "like the ones I know")
- γ is the *inverse width* of the bell: large γ, only very close points count
- The feature space of the RBF kernel has **infinitely many** dimensions, yet k costs one distance and one exponential

The Gram Matrix

The **Gram matrix** $K_{ij} = k(x_i, x_j)$ holds all the pairwise similarities of the training set. Example 1, RBF kernel with $\gamma = 0.5$:

	1	2	3	4	5
1	1.00	0.61	0.14	0.08	0.01
2	0.61	1.00	0.08	0.14	0.01
3	0.14	0.08	1.00	0.61	0.37
4	0.08	0.14	0.61	1.00	0.61
5	0.01	0.01	0.37	0.61	1.00

- $n \times n$, symmetric, ones on the diagonal; the blocks are the two groups (points 1-2 and 3-5)
- With broadcasting ($\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a^\top b$) it is three lines of `jax.numpy`, no loop

The Kernel SVM in Representer Form

The optimal f is a weighted sum of the similarities to the training points, with one weight β_j per point:

$$f(x) = \sum_{j=1}^{n} \beta_j \, k(x_j, x) + b$$

The loss is the same as before, written with K:

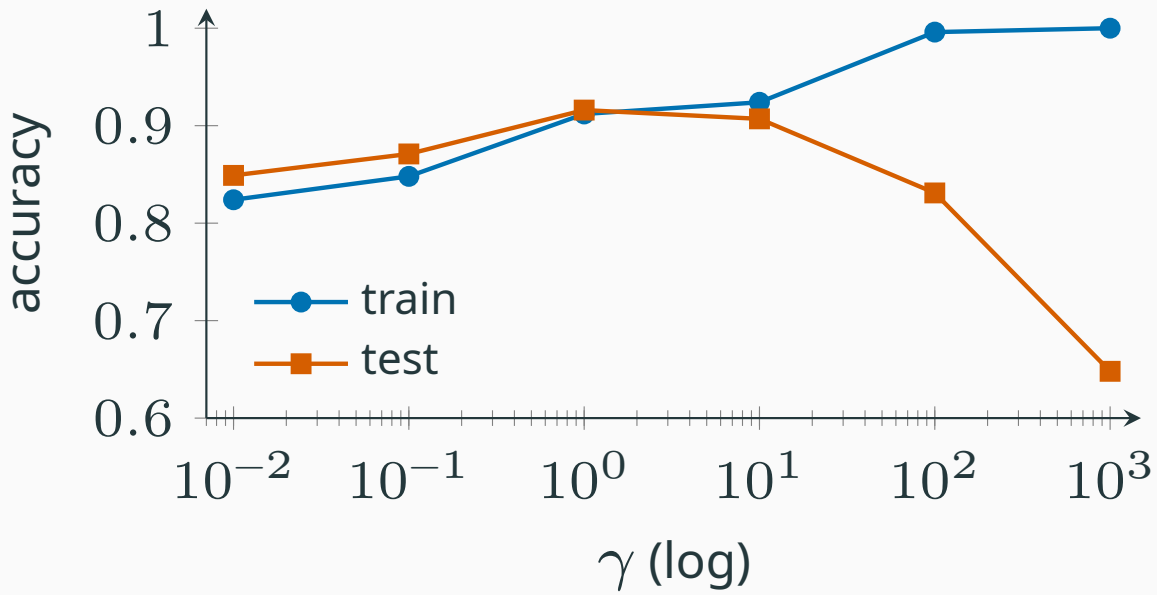
$$\mathcal{L}(\beta, b) = \frac{1}{n} \sum_i \max\left(0,\ 1 - y_i \left(K\beta + b\right)_i\right) + \frac{\lambda}{2} \beta^\top K \beta$$

- `jax.grad` gives the gradient with respect to β and b: no quadratic-programming solver
- The code agrees with scikit-learn's kernels to 10^{-15}

The Kernel SVM Against SVC

- The penalty $\beta^\top K \beta$ is $\|w\|^2$ **in the feature space**; no quadratic-programming solver is needed
- Lab moons data, $C = 10$, $\gamma = 1$: test accuracy ours **0.936**, SVC **0.935**, the same prediction on **99.8%** of the points
- The QP solver finds exactly-zero β_j outside the support vectors (12 of 300 on the ring, demo V3); gradient descent only makes them small: the *predictions* agree

γ: How Local Is the Similarity?



γ: Reading the Curves

- $\gamma = 0.01$**:** bells wider than the data: the boundary is almost a line, both accuracies are low (**underfitting**)
- $\gamma = 1$**:** the best test accuracy, 0.916
- $\gamma = 1000$**:** each point is its own island: training 1.000, test 0.648 (**overfitting**, 250 support vectors out of 250 points)
- The gap train $-$ test is the signature of overfitting (Class 01); C and γ together set the complexity

Which Kernel? (Demo V3)

Test accuracy, $C = 1$, scaled features:

kernel	moons	circles	breast cancer
linear	0.853	0.578	**0.965**
polynomial, degree 2	0.848	1.000	0.944
polynomial, degree 3	0.907	1.000	**0.965**
polynomial, degree 7	0.901	0.999	0.944
RBF	**0.913**	1.000	0.958
sigmoid	0.639	0.649	0.930

- The **linear** kernel is already best (or tied) on the 30 almost-linearly-separable measurements; curved boundaries pay on moons and circles
- The **RBF** kernel is the safe default: one parameter, works on any smooth problem
- A high degree adds nothing; the sigmoid kernel is not positive definite (avoid it)

Scaling Matters Even More Here

The RBF kernel depends on the **Euclidean distance**; a feature with a large range decides it.

breast cancer feature	range (max — min)
`mean_smoothness`	0.11
`mean_radius`	21
`mean_area`	2 357

Scaling Matters: the Result

input to the RBF SVM	test accuracy	grid-search best γ
raw features	0.916	10^{-6} (and slower)
standardized	**0.958**	10^{-2}

- Without scaling, only `mean_area` and `mean_perimeter` (hundreds to thousands) decide: the other 28 features are ignored
- The default `gamma="scale"` is $1/(d \cdot \mathrm{Var}(X))$: 6.3×10^{-7} raw, 3.3×10^{-2} standardized
- Rule: **standardize first** (training statistics only), then search C and γ

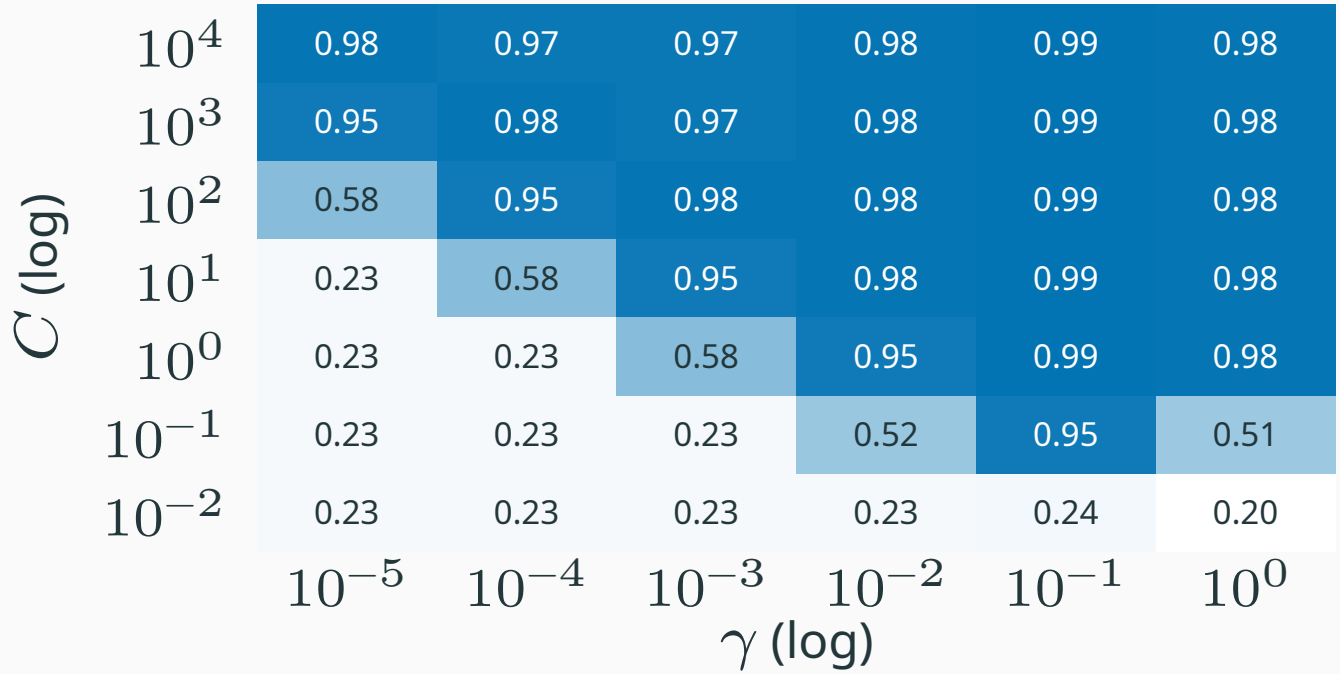
DEMO V3:

`01_svm.ipynb`

The SVM in Practice

Two Hyperparameters on a Log Grid

Cross-validated accuracy (5 folds), RBF SVM on the digits; C from 10^{-2} to 10^{4}, γ from 10^{-5} to 1:



- A **diagonal band** is good: a larger C needs a smaller γ
- The default ($C = 1$, `gamma="scale"`) is in the band: test 0.987 against 0.989 for the best cell ($C = 10$, $\gamma = 0.1$); 42 pairs $\times$ 5 folds took 6 s

The Best Grid Cell Is Optimistic

The best of 42 noisy scores is the maximum of 42 random numbers: **biased upwards**, even with cross-validation.

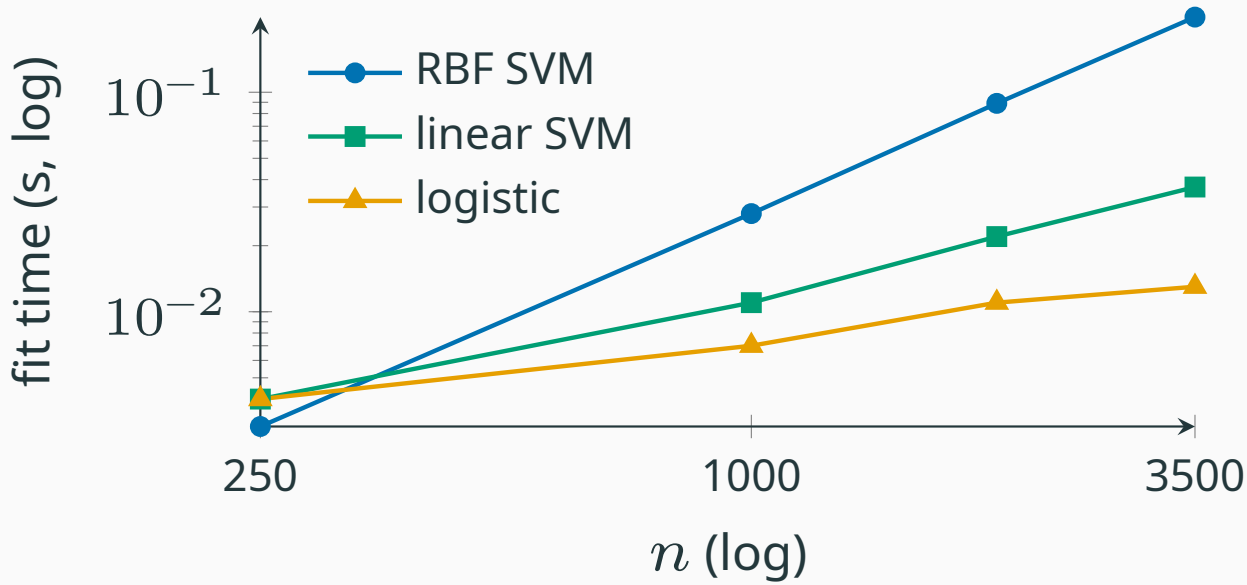
- **Nested cross-validation** (Class 01): the inner loop searches the grid, the outer loop measures the result on data that never influenced the choice
- Breast cancer (569 examples), 10 repetitions:

estimate	accuracy
best cell of the grid (inner CV only)	0.9793 ± 0.0032
nested cross-validation	0.9770 ± 0.0035
optimism of the naive number	**+0.0023**

- The bias is small here (a smooth, easy problem) and grows with the size of the grid and with *small* data sets
- Report the nested number, or a final test set used **once**

Cost: Time and Memory

Training solves a problem with n unknowns and an $n \times n$ Gram matrix. Spam data, 57 features, fit time against the number of examples:

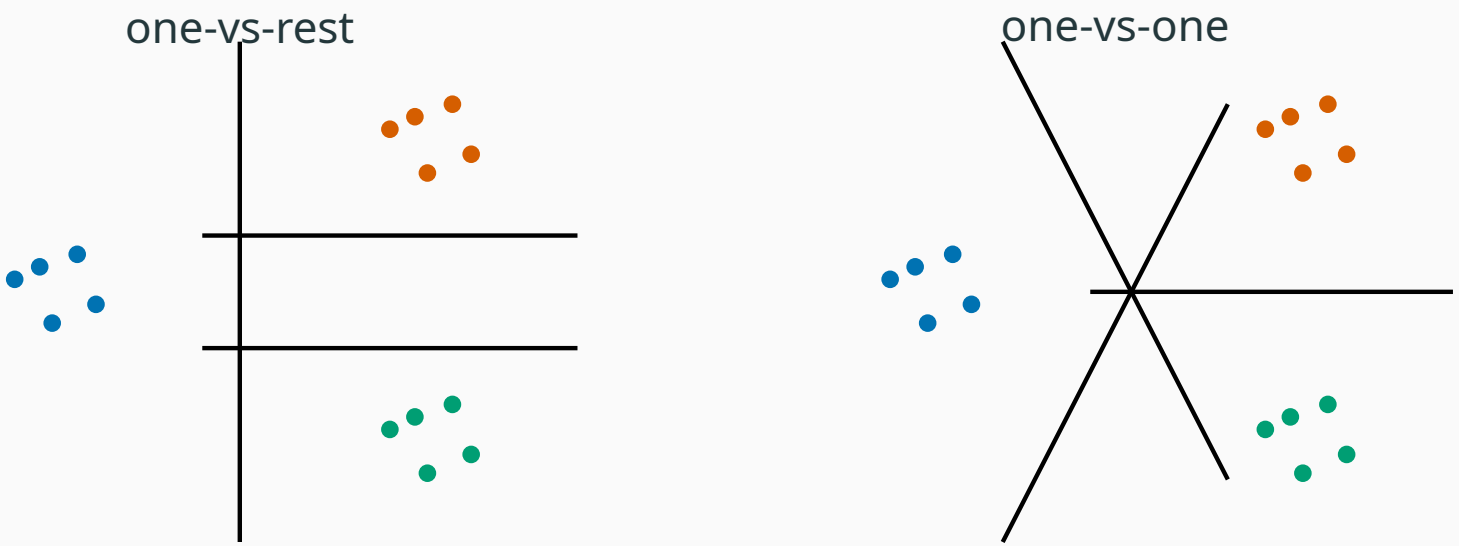


Cost: What It Means

- The RBF time grows with a log-log slope of **1.6** (between $n^{1.5}$ and n^3 in general); the linear models grow almost linearly
- The number of support vectors grows with n (143 at $n = 250$, 906 at $n = 3\,500$): *prediction* also gets slower, one kernel evaluation per support vector
- The Gram matrix of $100\,000$ examples in float64: $10^{10} \times 8$ bytes $= 80$ GB: kernel SVMs reach tens of thousands of examples, not millions
- Beyond that: linear SVM, or a neural network (next section)

More Than Two Classes

The SVM separates two classes; for K classes combine binary SVMs (here $K = 3$, three groups):



strategy	models	each model sees	digits ($K = 10$)
one-vs-rest	K	class k against all the rest	10 models, accuracy **0.991**
one-vs-one	$K(K-1)/2$	only the examples of two classes	45 models, accuracy **0.989**

One-vs-One and One-vs-Rest: Remarks

- One-vs-one: each model is *small* (two classes) and prediction is a **vote** of the 45 models; SVC does this internally
- One-vs-rest: models are *big* (all examples), but there are fewer
- Digits: 5 errors in 450 test images; the most frequent confusion is a 3 read as a 7
- Compare Class 03: the multinomial logistic regression learns the K classes **jointly** (softmax)

Probabilities and Unequal Costs

The SVM returns a **score** $f(x)$ (a signed distance), not a probability. Class 04 tools apply:

spam data, test	accuracy	log-loss	Brier
SVM, $\sigma(f)$ taken as probability	0.950	0.270	0.067
SVM, **Platt scaling** (a sigmoid fitted on held-out scores)	0.950	**0.147**	**0.038**
logistic regression	0.942	0.165	0.044

Unequal Costs: `class_weight`

- The ranking is **the same** (accuracy unchanged), but the numbers are calibrated only after Platt scaling
- **Costs** (German credit, a missed bad credit costs 5 refusals): give the expensive class a larger C (`class_weight`):

`class_weight`	accuracy	bad accepted	good refused	cost (5:1)
none	0.744	50	14	264
bad credit $\times 5$	0.716	20	51	**151**

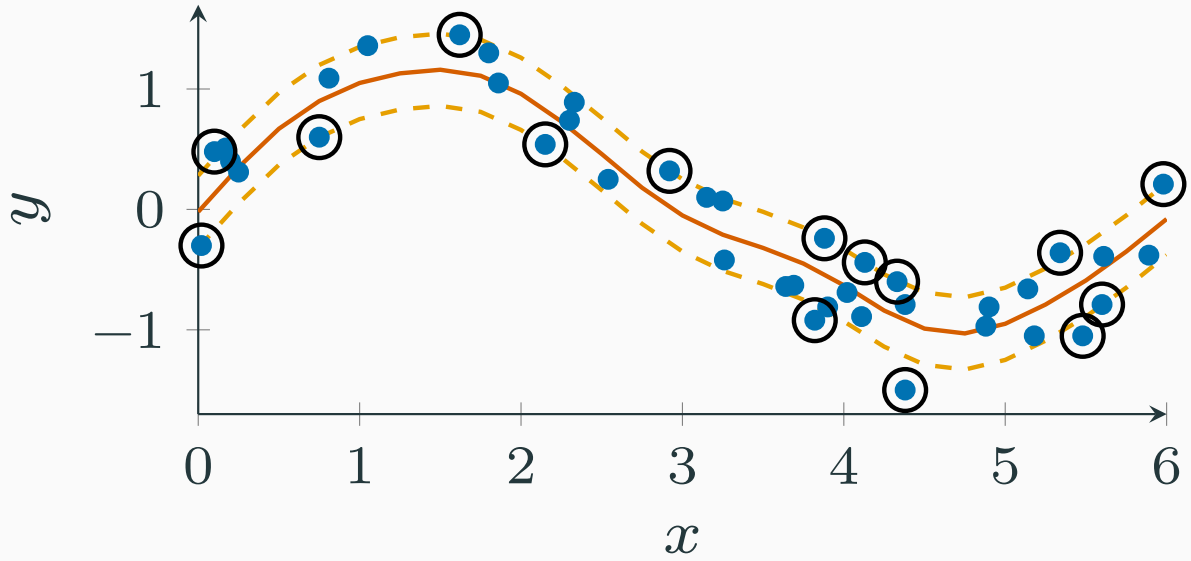
- Accuracy fell and the cost fell by 43%: the metric must follow the problem (Class 03)

Support Vector Regression: a Tube

For a **number** the margin becomes a *tube* of half-width ε around the prediction; errors inside it are free. The loss is

$$\ell_\varepsilon(r) = \max\big(0,\ |r| - \varepsilon\big), \qquad r = y - f(x)$$

and the problem is $\frac{1}{n}\sum_i \ell_\varepsilon(y_i - f(x_i)) + \frac{\lambda}{2}\|w\|^2$ (kernels work exactly as before).



Reading the Tube

- A noisy sinusoid, RBF kernel, $\varepsilon = 0.3$: **15 of 40** points are support vectors: those on or outside the tube
- The points *inside* do not influence the fit; ε plays the role of the noise level you accept

ε	0	0.1	0.3	0.6
support vectors (of 80)	80	50	19	3

SVR and Outliers

The ε-insensitive loss grows **linearly** (like the absolute error): a gross outlier has a bounded pull, unlike the squared error.

line $y = 2x + 1$ with three outliers (+25, +30, +28)	slope	intercept
least squares	2.02	**2.89**
SVR, linear, $\varepsilon = 0.5$	1.98	**1.22**

- The true intercept is 1: the three outliers pull least squares by 1.9, the SVR by 0.2
- Huber regression (Class 03 baselines) gives 1.23: the same idea of a bounded pull

SVR on Real Data

Concrete strength (1030 mixtures, 8 features), 5-fold RMSE in MPa:

model	RMSE (MPa)
mean	16.70
ridge	10.43
SVR, linear	10.76
SVR, RBF (default)	6.64
SVR, RBF ($C = 30$, $\gamma = 0.2$, $\varepsilon = 0.05$)	**5.90**

- The relation is **non-linear**: the kernel nearly halves the error; tuning gains another 11%
- Scale the **target** too (and give ε in its units)

DEMO V4–V5:

`01_svm.ipynb`

Neural Networks

Why Networks? The Missing Piece: the Features

Every model so far had its features **given**: the raw measurements, or polynomials, or the similarities of a kernel *chosen by us*.

model	where do the features come from?
linear / logistic regression	the raw measurements (or hand-made polynomials)
kernel SVM	a kernel we choose (γ, degree…)
neural network	**learned from the data**, together with the classifier

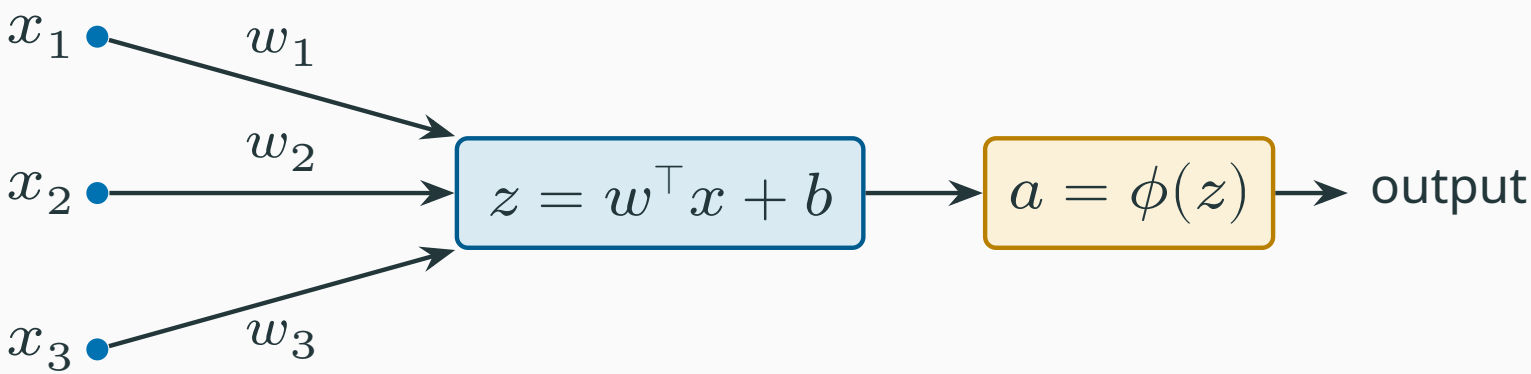
What a Hidden Layer Does

- A network is a model with **several stages**: each stage computes new features from the previous ones, and the last stage is a linear model on them
- All stages are trained *at once*, by gradient descent on one loss (Class 02): the features that help the loss are the ones that survive
- Fields that need this: images, sound, text, signals: no human knows the right features in advance (this is what *deep* learning scaled up)

The Neuron

A neuron is the model we already know, followed by a **non-linear function** ϕ (the *activation*):

$$a = \phi(w^\top x + b)$$



- ϕ = identity: **linear regression**; ϕ = sigmoid: **logistic regression**; ϕ = step: the **perceptron** (Class 03)
- So a neuron is not new: **the new thing is to connect many of them in layers**
- The weights w and the bias b are learned; ϕ is chosen

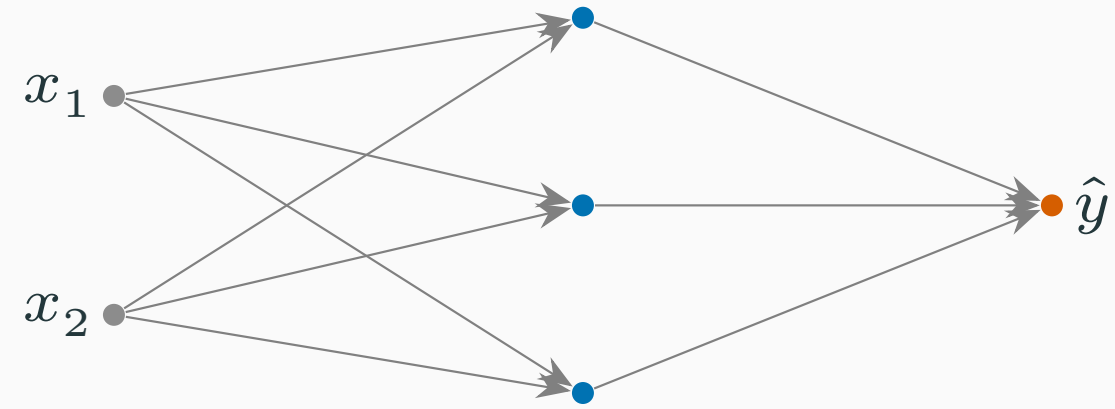
Layers: Many Neurons at Once

A **layer** is m neurons that read the same n inputs. Stack the weight vectors as the rows of a matrix W ($m \times n$): the whole layer is one matrix product,

$$a = \phi(Wx + b), \qquad W \in \mathbb{R}^{m \times n}, \quad b \in \mathbb{R}^m$$

- Parameters of a layer: $m \cdot n + m$. Example, **2 inputs** $\rightarrow$ **2 neurons**: W is 2×2 and b has 2 entries: 6 parameters
- A **multilayer perceptron (MLP)** feeds the output of one layer to the next:
  $x \rightarrow h = \phi(W_1 x + b_1) \rightarrow \hat{y} = W_2 h + b_2$

A Network with One Hidden Layer



- This network (2-3-1) has $2 \cdot 3 + 3 + 3 \cdot 1 + 1 = 13$ parameters; each hidden neuron is one *learned feature*

Why the Activation Must Be Non-Linear

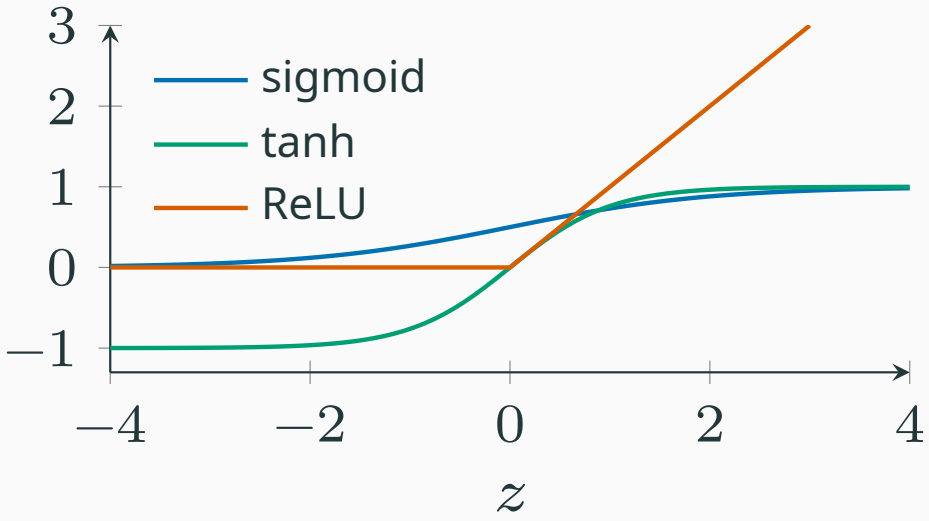
Without ϕ (or with ϕ = identity) the layers collapse:

$$W_2\left(W_1 x + b_1\right) + b_2 = \underbrace{\left(W_2 W_1\right)}_{W} x + \underbrace{\left(W_2 b_1 + b_2\right)}_{b}$$

Linear Activations in Practice

- Two linear layers **are one** linear layer: depth would add nothing, only parameters
- With a non-linear ϕ between them the composition is **not** a linear function: each hidden layer can bend the space
- Check on the digits: a 2-layer network with a *linear* activation scores **0.962**, logistic-regression level; with `tanh` **0.966**, `ReLU` 0.959, `sigmoid` only 0.900 (it is slower to train)
- On the two moons the linear-activation network scores 0.861 against 0.859 for logistic regression: the same model; `tanh`/`ReLU` reach 0.93

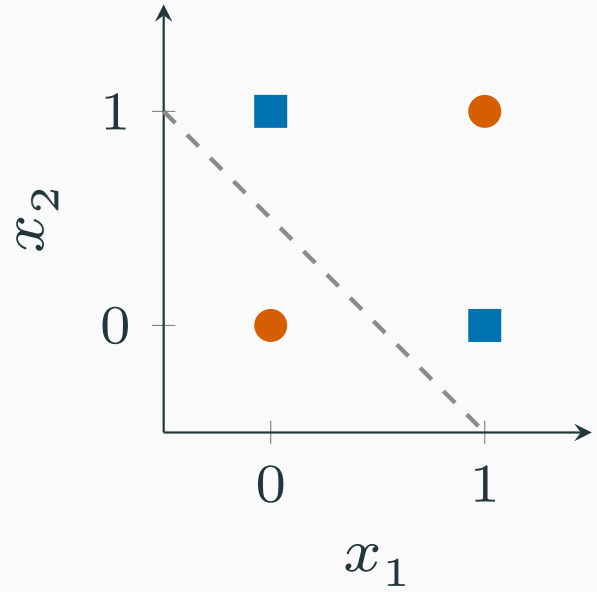
Activation Functions



- **ReLU** $= \max(0, z)$: zero or the identity; derivative 0 or 1; the default for hidden layers
- **Sigmoid** $\in (0, 1)$, derivative ≤ 0.25: for the output of a probability
- **tanh** $\in (-1, 1)$, derivative ≤ 1: older hidden layers
- **Softmax** turns K scores into K probabilities that sum to 1: the output for K classes

XOR: No Line Separates It

Example 3: XOR is 1 when exactly one input is 1.



- No line puts the two blue points on one side and the two red points on the other (Class 03)
- The best a line can do is three of the four points (dashed): accuracy 0.75
- A hidden layer of two ReLU neurons solves it: next frame, by hand

XOR, Solved by Hand

One hidden layer of two ReLU neurons, then a linear output:

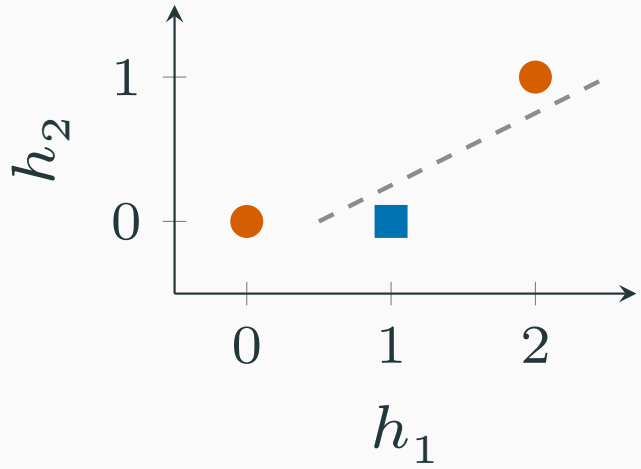
$$h_1 = \mathrm{ReLU}(x_1 + x_2), \qquad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$$

$$\text{output} = h_1 - 2\,h_2$$

x_1	x_2	h_1	h_2	output	target
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	2	1	**0**	0

XOR: What the Hidden Layer Does

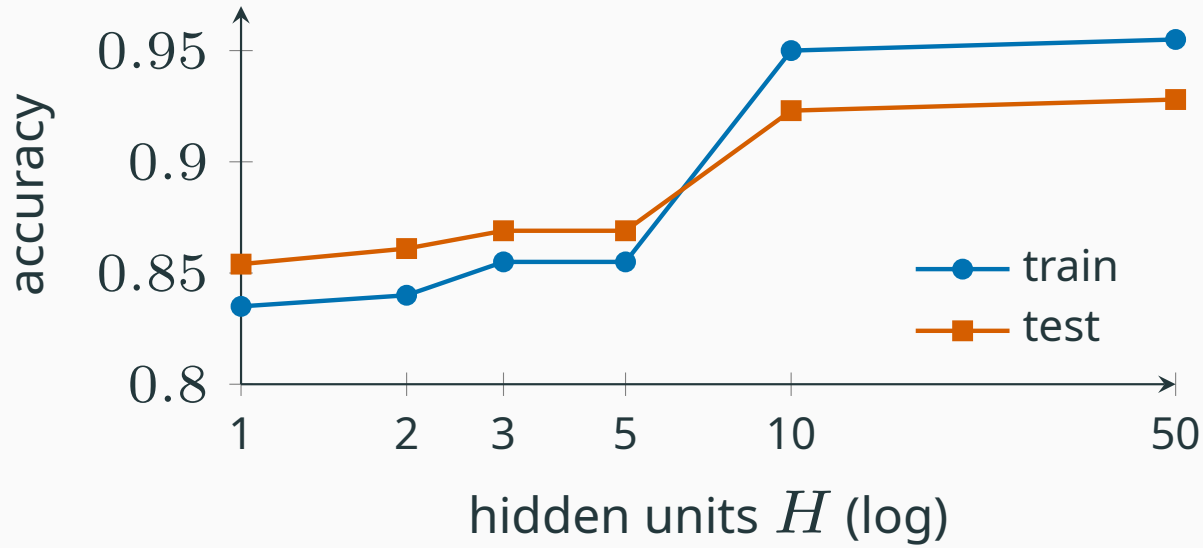
The hidden layer **re-describes** the points: $(1, 1)$ goes to $(h_1, h_2) = (2, 1)$, where a line works.



- The two blue points land on the **same** place $(1, 0)$; the dashed line is $h_1 - 2h_2 = 0.5$
- On the continuous XOR of Class 03, a network with no hidden layer reaches **0.733**; with 8 hidden units **1.000**: training finds such weights, nobody wrote them
- The hidden units are *features*: here something like $|x_1 - x_2|$

How Many Hidden Units? (Demo H1)

Two moons (200 points, noise 0.25), one hidden layer of H ReLU units, median of 3 runs:



- Parameters: $H = 1$: 5, $H = 10$: 41, $H = 50$: 201. Each ReLU neuron adds a **kink**: the boundary is *piecewise linear* with about H pieces
- With one unit the network is a linear classifier; the big jump comes at 10 units; beyond that little (the noise limits the data)

Universal Approximation: What It Says

Theorem (Cybenko 1989, Hornik 1991): a network with **one** hidden layer and enough units can approximate any continuous function on a bounded region, to any accuracy.

Fit of $\sin(2x)$ on $[-3, 3]$ by one hidden layer of ReLU units (400 epochs of Adam):

hidden units	2	5	20	100
parameters	7	16	61	301
RMSE	0.556	0.306	0.308	**0.092**

Universal Approximation: the Limits

- **What it does not say:**
  - how *many* units (possibly astronomically many)
  - that gradient descent **finds** the weights: 5 and 20 units gave the same error, the optimizer was the limit
- Why depth then? Several small layers can represent some functions with exponentially fewer units than one wide layer, and compose features (edges $\rightarrow$ shapes $\rightarrow$ objects)
- Existence is not learnability: generalization still needs data and regularization (Class 01)

The Output Layer Follows the Task

The hidden layers are the same; the last layer and the loss follow the task, as in Class 04 (the loss is the **negative log-likelihood** of an assumption about y):

task	output layer	loss	assumption
regression	1 unit, identity	mean squared error	Gaussian
two classes	1 unit, sigmoid	binary cross-entropy	Bernoulli
K classes	K units, softmax	categorical cross-entropy	categorical

Softmax and Its Gradient

- **Softmax** turns K scores z into probabilities: $p_k = e^{z_k} / \sum_j e^{z_j}$
  - Example $z = (2.0,\ 1.0,\ 0.1)$: $p = (0.659,\ 0.242,\ 0.099)$
- True class = the first one: loss $-\log 0.659 = 0.417$
- Gradient with respect to the scores: $p - \text{onehot} = (-0.341,\ 0.242,\ 0.099)$
  - *prediction minus target*, the same residual as in linear and logistic regression
- A network with 10 untrained classes starts at $-\log \frac{1}{10} = 2.303$: a sanity check

DEMO H1:

`02_neural_networks.ipynb`

Backpropagation

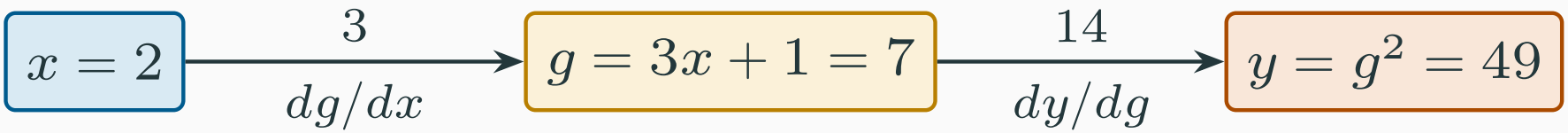
What Training Needs: the Gradient of Every Weight

Gradient descent (Class 02) needs $\partial \mathcal{L} / \partial \theta$ for **every** parameter: thousands to billions of numbers, at every step.

- **Finite differences:** change one weight by 10^{-4} and re-run the network
  - two forward passes **per parameter**: $4\,820$ for the digits network ($2\,410$ parameters)
- **Backpropagation:** one forward pass, one backward pass that gives **all** the derivatives, for about twice the cost of the forward pass
  - it is the **chain rule**, organized to reuse what the forward pass computed (Rumelhart, Hinton, Williams, 1986)
- In practice `jax.grad` (Class 02) and Keras do it; we derive it **once**, by hand, to know what they do

The Chain Rule in One Slide

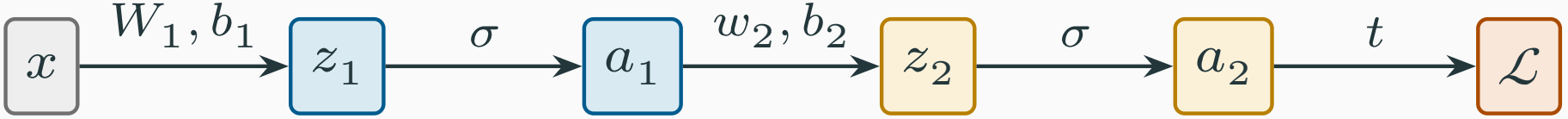
If $y = f(g(x))$ then $\dfrac{dy}{dx} = \dfrac{dy}{dg} \cdot \dfrac{dg}{dx}$: **multiply the local slopes along the path**.



- $\dfrac{dy}{dx} = 14 \cdot 3 = 42$; check: $y = (3x+1)^2$, $y' = 6(3x+1) = 42$ ✓
- With several paths (a variable used twice) the contributions **add**
- A network is a long chain $x \to z_1 \to a_1 \to z_2 \to a_2 \to \mathcal{L}$:
  - the derivative of the loss with respect to a first-layer weight is a **product of local slopes** along the chain

A Tiny Network: the Forward Pass

2 inputs, 2 sigmoid hidden units, 1 sigmoid output, one example, squared error $\mathcal{L} = \frac{1}{2}(a_2 - t)^2$:



$$x = (1,\ 0.5),\ W_1 = \begin{pmatrix} 0.1 & 0.4 \\ -0.2 & 0.3 \end{pmatrix},\ b_1 = (0,\ 0.1),$$
$$w_2 = (0.7,\ -0.5),\ b_2 = 0.2,\ t = 1$$

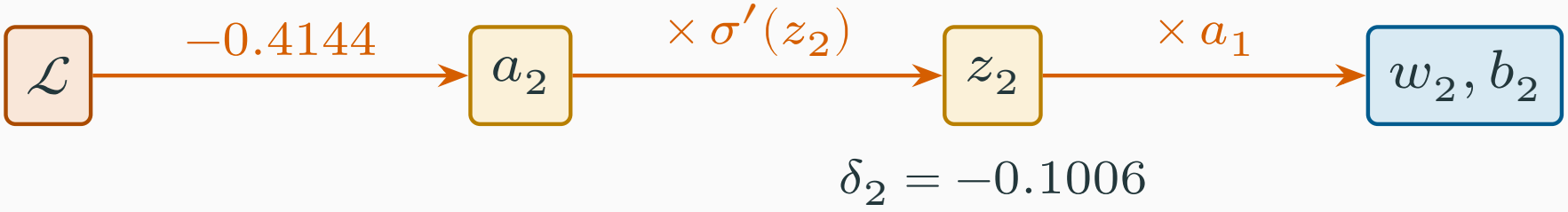
The Forward Pass: Numbers

- $z_1 = (0.1 \cdot 1 + 0.4 \cdot 0.5,\ -0.2 \cdot 1 + 0.3 \cdot 0.5 + 0.1) = (0.30,\ 0.05)$
- $a_1 = \sigma(z_1) = (0.5744,\ 0.5125)$
- $z_2 = 0.7 \cdot 0.5744 - 0.5 \cdot 0.5125 + 0.2 = 0.3459$
- $a_2 = \sigma(z_2) = 0.5856$
- $\mathcal{L} = \frac{1}{2}(0.5856 - 1)^2 = \mathbf{0.0859}$

The prediction (0.586) is far from the target (1): the backward pass finds how to change every weight to lower $\mathcal{L}$.

The Backward Pass: the Output Layer

Walk the chain from the loss back. The **error signal** is $\delta = \partial \mathcal{L} / \partial z$ of each layer; the sigmoid has $\sigma' = a(1-a)$.

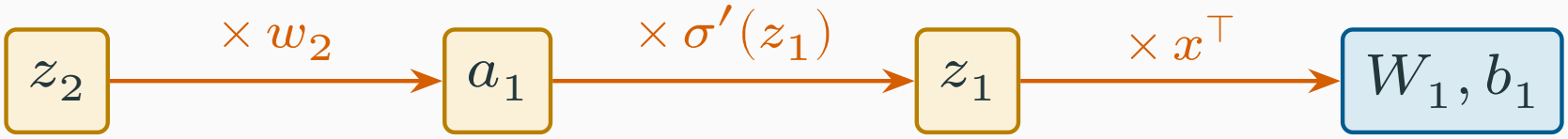


1. **Output error:** $\dfrac{\partial \mathcal{L}}{\partial a_2} = a_2 - t = -0.4144,$

   $\sigma'(z_2) = 0.5856 \cdot 0.4144 = 0.2427$
2. So $\delta_2 = -0.4144 \cdot 0.2427 = -\mathbf{0.1006}$
3. **Output weights:**

   $$\frac{\partial \mathcal{L}}{\partial w_2} = \delta_2 \, a_1 = (-0.0578,\ -0.0515), \quad \frac{\partial \mathcal{L}}{\partial b_2} = \delta_2$$

The Backward Pass: the Hidden Layer



$\delta_2 = -0.1006$ $\delta_1 = (-0.0172,\ 0.0126)$

4. **Send the error back** through w_2 and the hidden sigmoids (element-wise product $\odot$):

$$\delta_1 = (w_2\,\delta_2) \odot a_1(1 - a_1)$$

$$\delta_1 = (-0.0704,\ 0.0503) \odot (0.2445,\ 0.2498)$$

$$\delta_1 = (-0.0172,\ 0.0126)$$

The Backward Pass: the Hidden Weights

5. **Hidden weights:**

$$\frac{\partial \mathcal{L}}{\partial W_1} = \delta_1 \, x^\top = \begin{pmatrix} -0.01721 & -0.00860 \\ 0.01256 & 0.00628 \end{pmatrix}, \qquad \frac{\partial \mathcal{L}}{\partial b_1} = \delta_1$$

- Each step reuses numbers **already computed** in the forward pass (a_1, a_2): that is the saving
- Next frame: the same four numbers from `jax.grad` and from finite differences

Check: by Hand, `jax.grad`, Finite Differences

parameter	by hand	`jax.grad`	finite differences ($\pm 10^{-4}$)
$\partial \mathcal{L} / \partial w_{2,1}$	-0.05777	-0.05777	-0.05767
$\partial \mathcal{L} / \partial b_2$	-0.10056	-0.10056	-0.10055
$\partial \mathcal{L} / \partial W_{1,11}$	-0.01721	-0.01721	-0.01717
$\partial \mathcal{L} / \partial W_{1,21}$	0.01256	0.01256	0.01248

- Hand and `jax.grad` agree to 5 digits; finite differences to 3 digits (their own error)
- One gradient step with learning rate 2 and 60 repeats: the loss of this example goes $0.0859 \to 0.056 \to 0.019$ (5 steps) $\to 0.0043$ (20) $\to$ **0.0013** (60)
- **Debugging tip:** when you write a layer by hand, compare its gradient with finite differences first

Backpropagation in Matrix Form

For layers $z_\ell = W_\ell\, a_{\ell-1} + b_\ell,\ a_\ell = \phi(z_\ell),\ \ell = 1, \dots, L$:

1. **Output error:** $\delta_L = \dfrac{\partial \mathcal{L}}{\partial a_L} \odot \phi'(z_L)$ (softmax with cross-entropy: simply $p - \text{onehot}$)
2. **One layer back:** $\delta_\ell = (W_{\ell+1}^\top\, \delta_{\ell+1}) \odot \phi'(z_\ell)$
3. **Weights:** $\dfrac{\partial \mathcal{L}}{\partial W_\ell} = \delta_\ell\, a_{\ell-1}^\top$

and $\dfrac{\partial \mathcal{L}}{\partial b_\ell} = \delta_\ell.$

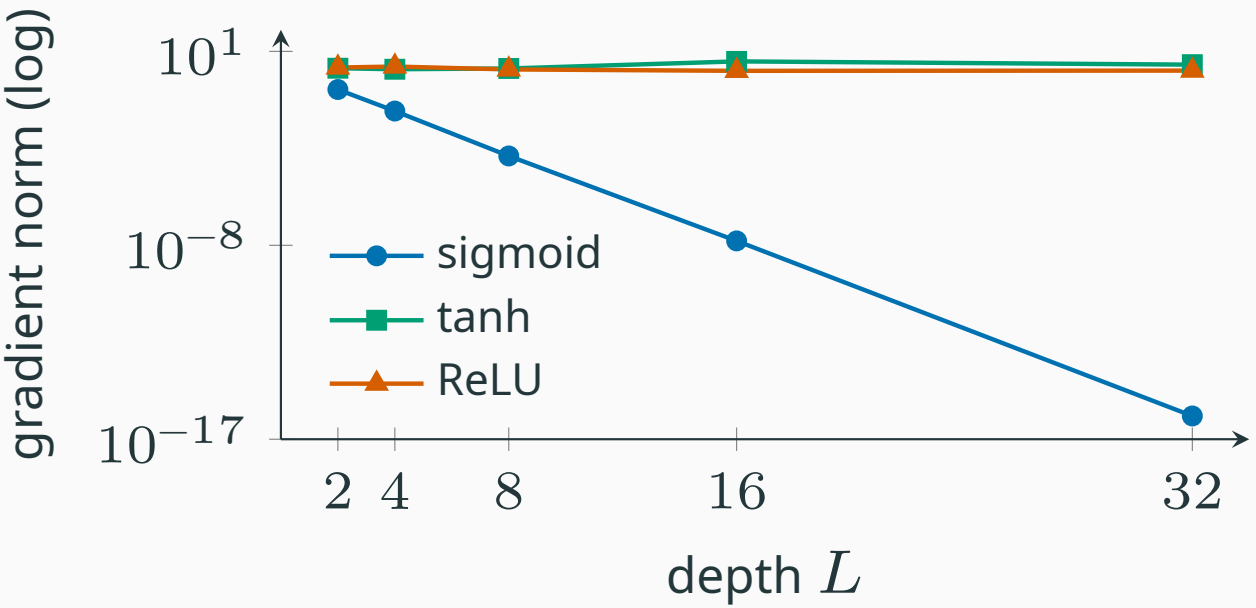
Backpropagation: Remarks

- For a minibatch, δ and a are matrices (one row per example) and the products are sums over the batch, divided by n
- Step 2 is the heart: the error is carried **backwards by the transpose** of the weights, and multiplied by the local slope of the activation
- This is what Lab Part E asks you to write for a ReLU network ($\phi'(z) = [z > 0]$) and to check against `jax.grad`: the difference is 10^{-17}

Vanishing Gradients

Step 2 multiplies by $\phi'(z)$ at every layer. If $\phi' \leq 0.25$ (sigmoid) the gradient shrinks like 0.25^L over L layers: the first layers hardly learn.

Norm of the gradient of the **first** layer, MLP of width 32, median of 5 initializations:



Vanishing Gradients: Remedies

- Sigmoid: 15 orders of magnitude lost at depth 32. ReLU has slope 1 where active: the signal survives
- Remedies: ReLU-family activations, a good **initialization** (next section), normalization layers, residual connections
- The opposite problem, **exploding** gradients (slopes or weights > 1), is cured by the same tools plus gradient clipping

What Training Costs

The forward pass of a dense layer is one matrix product: about one multiply-add per weight; the backward pass about twice that.

hidden layers	parameters (64 inputs, 10 classes)
none (logistic regression)	650
[16]	1 210
[64, 64]	8 970
[256, 256]	85 002
[512, 512, 512]	563 722

Training Cost: What It Means

- One epoch of n examples costs about $3 \cdot n \cdot (\text{parameters})$ multiply-adds
- Memory: the forward values a_ℓ must be kept for the backward pass: *this* is what limits the batch size on large models
- The digits network of 8 970 parameters trains 60 epochs in about **2 seconds** on a laptop CPU

DEMO H2:

`02_neural_networks.ipynb`

Training Networks

The Training Loop

The recipe of Class 02 on a non-convex loss. One **step** = one gradient update on a **minibatch**; one **epoch** = one pass over the training data.

1. **Initialize** the weights (randomly)
2. **Repeat** for each epoch: shuffle the examples, cut them into minibatches of B
3. For each minibatch: forward pass $\rightarrow$ loss $\rightarrow$ backward pass $\rightarrow$ update (SGD, momentum, Adam)
4. After each epoch: measure the loss and the metric on the **validation** set; stop when it stops improving

Minibatches and Epochs: an Example

- Digits: 1 077 training images, $B = 32 \Rightarrow \lceil 1077/32 \rceil = 34$ updates per epoch; 60 epochs $=$ 2 040 updates
- Why minibatches?
  - all examples: *exact but slow*; one example: *fast but noisy*
  - a minibatch of 32-256 is a good compromise, and runs well on modern hardware
- The loss is **not convex**: many minima, saddle points; different starts give different networks (see the seeds)

Initialization

If all weights start equal, all hidden units compute the same value and get the same gradient: they stay identical forever (**symmetry**). Random values break it; their *scale* matters.

- **Glorot** (tanh, sigmoid): $\mathrm{Var}(W) = \dfrac{2}{n_{\text{in}} + n_{\text{out}}}$
- **He** (ReLU): $\mathrm{Var}(W) = \dfrac{2}{n_{\text{in}}}$
- Both keep the variance of the signal constant from layer to layer; the biases start at 0

Initialization: What Happens

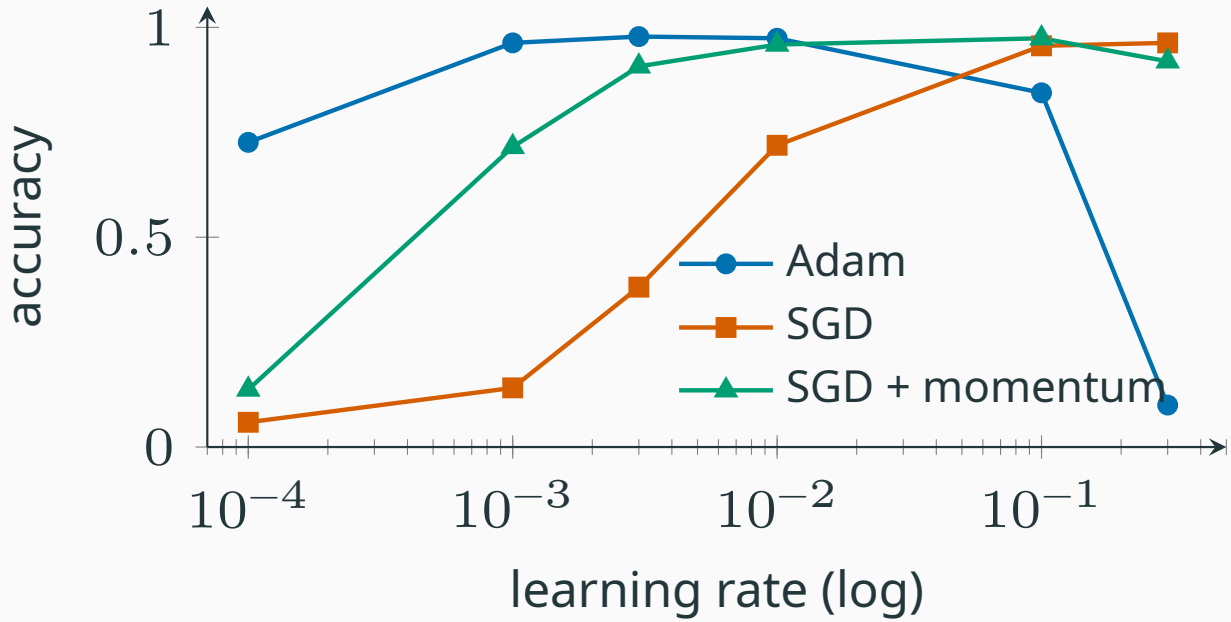
Digits, 2 hidden layers of 32 units:

initializer	loss, epoch 1	val. acc., epoch 10	distinct units
zeros	2.303	0.100	**1** (of 32)
normal, sd 0.01	2.300	0.670	32
normal, sd 1	**14.385**	0.330	32
Glorot uniform	2.150	**0.937**	32
He normal	2.130	0.922	32

- Too small: the signal vanishes and learning is slow; too large: a huge first loss and unstable training

The Learning Rate

The most important hyperparameter (Class 02). Validation accuracy of the digits network after 20 epochs:



The Learning Rate: Reading the Curves

- The **best learning rate of SGD is 100 times that of Adam**: a rate is only meaningful *with* its optimizer
- Search it on a **log grid**; too large gives chance level (0.100 = 1 in 10 classes), too small never arrives
- Adam is the default: it scales the step per weight (Class 02); SGD with momentum often generalizes slightly better on large problems, and needs more tuning

Batch Size

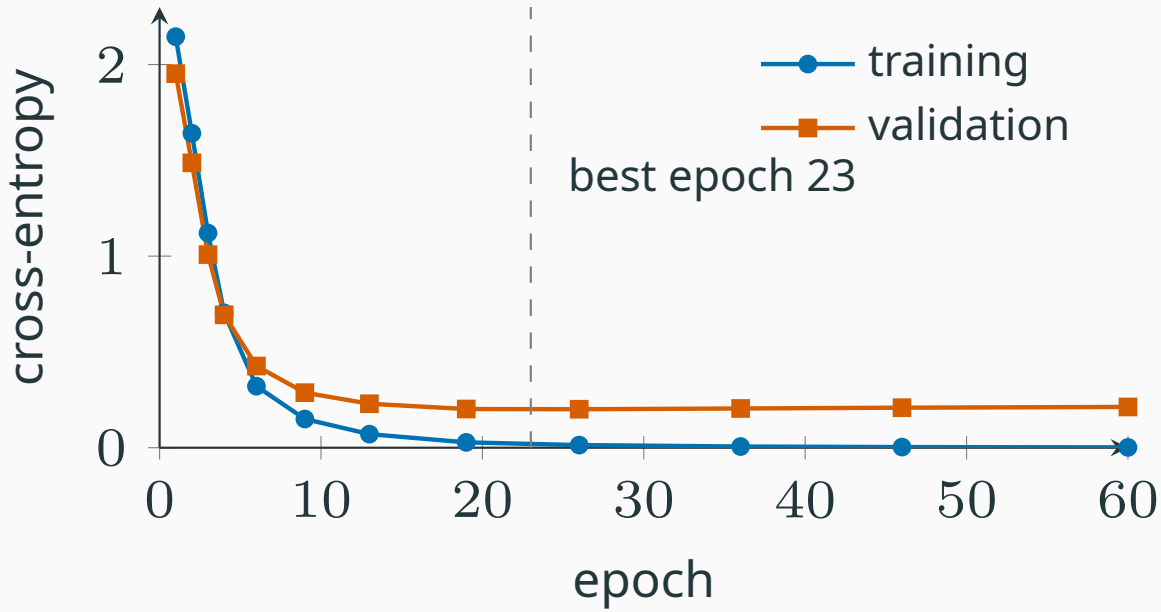
Digits network, Adam 10^{-3}, 5 epochs:

batch size	updates per epoch	validation accuracy after 5 epochs	time of 20 epochs (s)
8	135	**0.952**	1.0
32	34	0.907	0.7
128	9	0.785	0.8
512	3	0.444	0.7
1 077 (all)	1	0.181	0.3

- Per **epoch**, small batches make more progress: more updates; per **second**, large batches use the hardware better
- A larger batch needs a larger learning rate to make the same progress; $B = 32$ is the usual starting point
- Noise from small batches also acts as a mild regularizer

Overfitting a Network

A network with more parameters than examples *can* memorize the data: 300 digits, two hidden layers of 128 units (34 000 parameters).



Overfitting: Three Tools

- Training loss $\to 0.002$ (accuracy 1.000) while the validation loss turns up after epoch 23: 0.198 $\to$ 0.213 (**+7%**), the signature of overfitting (Class 01)
- Three tools follow: **weight decay**, **early stopping**, **dropout**

Weight Decay (the L_2 Penalty)

Add $\frac{\lambda}{2}\sum W^2$ to the loss, as in ridge (Class 03) and the Gaussian prior (Class 04). Each update **shrinks** the weights:

$$\theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\,\nabla\mathcal{L}$$

- Small weights give smooth functions: a large weight is what lets a network draw a sharp, memorizing boundary
- In Keras: `kernel_regularizer=keras.regularizers.L2(1e-3)`; the bias is usually not penalized

Weight Decay: the Result

- Hundred noisy moons (70 train), networks of 2×128 units, test accuracy (median of 3 seeds):

regularization	train	test
none	0.893	0.805
weight decay 10^{-3}	0.843	0.825
weight decay 10^{-2}	0.800	**0.832**

- The training accuracy **falls** and the test accuracy **rises**: the gap closes

Early Stopping

The number of epochs is a hyperparameter; let the **validation loss** choose it.

1. After every epoch, compute the validation loss
2. If it improved, remember the weights
3. If it has not improved for `patience` epochs, stop and **restore the best weights**

- Keras: `EarlyStopping(monitor="val_loss", patience=20, restore_best_weights=True)`
- Digits: stops at epoch **54** of 100, half of the time; test accuracy 0.973 against 0.969
- Fewer steps keep the weights near their small initial values: a regularizer too
- The validation set now *steers training*: keep a separate **test set** for the final number (Class 01)

Dropout

During training, switch off each hidden unit with probability p, a **different** random set for every minibatch.



- Example $p = 0.5$: the kept values are divided by $1 - p$, so the *expected* activation is unchanged
- No unit can count on a partner: each learns a feature that is useful on its own
- At test time **no** unit is dropped (Keras does it for you in `predict`)

Which Regularizer?

Noisy moons (70 training points, networks of 2×128), test accuracy, median of 3 seeds:

none	dropout 0.5	early stopping	weight decay 10^{-2}	logistic regression
0.805	0.824	0.826	0.832	**0.829**

- Honest reading: on this tiny noisy problem every regularizer brings the big network **back to the level of the linear model**; the data cannot support more

Seeds: Report the Spread

The initial weights and the minibatch order are random: the result of one run is a **random variable** (Class 01).

digits, 10 seeds, same data and settings	test accuracy
mean	0.973
standard deviation	0.004
minimum / maximum	0.964 / 0.978

- The best seed is not the model: report **mean** $\pm$ **sd** over seeds, or choose by validation and report the test once
- A difference of 0.5% between two configurations is *inside* this spread: do not conclude from one run
- Fix `keras.utils.set_random_seed(seed)` for reproducibility, and vary it on purpose to measure the spread

Networks in Keras

The network of the digits (Keras 3, JAX backend: `export KERAS_BACKEND=jax`):

```
L = keras.layers
model = keras.Sequential([
    keras.Input((64,)),
    L.Dense(64, activation="relu"), L.Dropout(0.2),
    L.Dense(64, activation="relu"),
    L.Dense(10, activation="softmax"),
])
model.compile(keras.optimizers.Adam(1e-3),
              "sparse_categorical_crossentropy")
model.fit(X, y, epochs=100, batch_size=32,
          validation_split=0.2)
```

- `Dense(64, activation="relu")` is ReLU$(Wx + b)$; `compile` chooses the loss and the optimizer; `fit` is the training loop
- The same model in JAX takes about 20 lines (lab Part E): 0.976 against 0.973 for Keras

A Network for Regression (Demo H5)

Output layer: **one linear unit**; loss: **mean squared error**; scale the **target** (and the features).

5-fold RMSE	mean	ridge	SVR (RBF)	MLP [32]	MLP [64, 64]
concrete (MPa)	16.70	10.43	5.90	6.09	**4.83**
abalone (rings)	3.22	2.21	2.14	**2.09**	2.10
auto-mpg	7.79	3.38	**2.60**	2.71	2.67
diabetes	76.98	**54.78**	59.14	56.35	65.44

Regression: Reading the Results

- A network helps when there is **non-linear structure and enough data**: concrete (non-linear mixtures) halves the linear error; **diabetes** (442 noisy rows) does not benefit: the larger network is *worse*
- California housing (20 640 rows): ridge 0.671, MLP $[64, 64]$ **0.537**, MLP 3×128 0.524, SVR on 3 000 rows 0.574 (the SVR cannot use all rows)
- **Target scale:** training on raw MPa values the validation RMSE is 15.5 after 5 epochs, against 8.0 with a standardized target; both end near 5.3

DEMO H3–H5:

`02_neural_networks.ipynb`

Choosing a Learner

Linear Models, SVM and Neural Networks

	logistic / linear	SVM (RBF)	MLP
features	given	via a kernel	**learned**
loss	convex	convex	**non-convex**
solution	unique	unique	depends on the start
hyperparameters	λ	C, γ	width, depth, rate, λ
scaling of features	needed	**essential**	needed
training cost	$O(n)$	$O(n^2$–$n^3)$	$O(n \cdot p)$ per epoch
large n	easy	hard (memory)	**easy**
interpretable	weights	support vectors	hard

Which Learner First?

- Small, tabular data: all three are close; **start with the linear model** and the SVM
- Many examples, images, text, signals: networks

One Protocol, Five Data Sets (Demo H6)

Median test accuracy, 5 stratified 75/25 splits, standardization on the training part:

data set	logistic	SVM (RBF)	MLP
breast cancer	**0.972**	0.965	0.969
spam	0.941	**0.952**	**0.952**
digits	0.969	**0.982**	0.972
wine (quality ≥ 6)	0.730	**0.779**	0.768
heart disease	**0.849**	0.816	0.836

- **No learner wins everywhere** (No Free Lunch, Class 01): the best is the SVM on three, the linear model on two
- On small, near-linear data (breast cancer, heart disease) the differences are *inside the noise* (sd 0.01–0.05)
- Cost per fit: logistic 0.01 s, SVM 0.1–0.7 s, MLP 1–2 s

Is a Difference Real? The Corrected t-Test

- Repeated random splits share training data: the J scores are **correlated**
- The usual paired t-test then has too small a variance and finds differences that are not there
- Nadeau and Bengio (2003) correct it, with d_j the score difference on split j:

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\right) s_d^2}}, \qquad J - 1 \text{ degrees of freedom}$$

The Corrected Test: the Results

Difference of mean accuracy (and p-value), 5 splits:

data set	SVM — MLP	SVM — logistic	MLP — logistic
spam	-0.004 (0.27)	$+0.009$ (0.058)	$+0.013$ (0.065)
digits	$+0.009$ (0.014)	$+0.012$ (0.09)	$+0.004$ (0.46)
wine	$+0.012$ (0.033)	$+0.046$ ($<$0.001)	$+0.035$ ($<$0.001)

- Only the **wine** data show a clear gain of the non-linear learners over the linear one; elsewhere the evidence is weak with 5 splits
- Significant is not *important*: 0.9% on digits is small; weigh it against cost and simplicity

Key Takeaways (1/2)

- **SVM:** the widest-margin line; defined by the **support vectors**; soft margin = hinge loss $+ L_2$, $C = 1/(\lambda n)$
- **Kernels:** the SVM needs only dot products: replace them by a similarity $k(a, b)$
  - curved boundaries without building features
  - choose C and γ on a **log grid**, after **scaling**; cost $O(n^2)$ at least
- **SVR:** a tube of half-width ε and a loss that grows linearly: robust to outliers

Key Takeaways (2/2)

- **Neural network:** neurons (a linear model plus a non-linearity) in layers; the hidden layers **learn the features**; the output layer and the loss follow the task
- **Backpropagation:** the chain rule backwards, $\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot \phi'(z_\ell)$; `jax.grad` and Keras do it
- **Training:** a good initialization, an adaptive optimizer on a log grid of rates, minibatches, **weight decay, early stopping, dropout**, several seeds
- **Choice:** no learner wins everywhere; compare on the same protocol with a corrected test

Lab 05: Build It in JAX and Keras

Part	Task	twin
A	Data visualization and preparation	—
B	Linear SVM: hinge loss with `jax.grad`, C	`SVC(kernel="linear")`
C	Kernels, kernel SVM, γ, the grid	`SVC`, `GridSearchCV`
D	The ε-insensitive loss	`SVR`
E	A small network in JAX, backpropagation by hand	`jax.grad`, `MLPClassifier`, Keras
F	Keras: regularization, early stopping, seeds	`MLPClassifier`
G	Challenge: which learner?	`LogisticRegression`, `SVC`, Keras

Lab 05: How It Connects

- Guide `practice/05_svm_neural_networks.pdf`, notebook `lab05_svm_neural_networks.ipynb`, solution in `solutions/`
- Build the **SVM** from its loss and compare it with SVC; build a **small network** in JAX and compare it with `MLPClassifier` and a Keras network (JAX backend)
- **Project 1:** the SVM and the network are your *analogizer* and *connectionist*: standardize, search on a log grid inside the cross-validation, and report the spread

References (1/2)

- C. Cortes, V. Vapnik, “Support-vector networks”, *Machine Learning*, 1995
- B. Schölkopf, A. Smola, *Learning with Kernels*, MIT Press, 2002
- C. Bishop, *Pattern Recognition and Machine Learning*, 2006: chapters 5 (neural networks), 6 and 7 (kernels, sparse kernel machines)
- G. James et al., *An Introduction to Statistical Learning with Applications in Python*, 2023: chapters 9 (SVM) and 10 (deep learning)
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, 2016: chapters 6 (feedforward networks), 7 (regularization), 8 (optimization)

References (2/2)

- D. Rumelhart, G. Hinton, R. Williams, “Learning representations by back-propagating errors”, *Nature*, 1986
- K. Hornik, “Approximation capabilities of multilayer feedforward networks”, *Neural Networks*, 1991
- X. Glorot, Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks”, *AISTATS*, 2010; K. He et al., “Delving deep into rectifiers”, *ICCV*, 2015
- N. Srivastava et al., “Dropout”, *JMLR*, 2014; D. Kingma, J. Ba, “Adam”, *ICLR*, 2015
- C. Nadeau, Y. Bengio, “Inference for the generalization error”, *Machine Learning*, 2003
- J. Platt, “Probabilistic outputs for support vector machines”, 1999